Character Animation

Ludovic Hoyet (Inria)

Context

- Goal: make "*things*" move
  - Approches valid for character animation

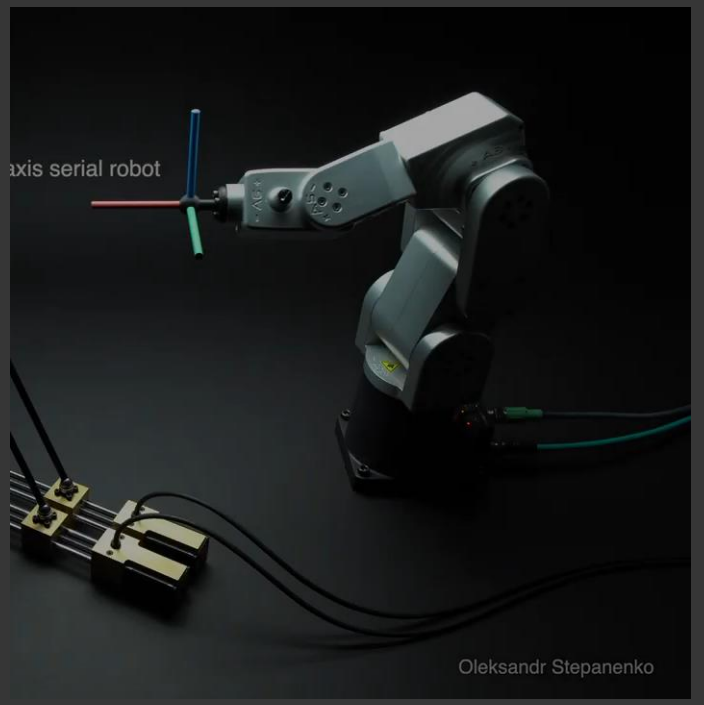
?

For Honor (Ubisoft)

Context

- Goal: make "*things*" move
  - Approches valid for character animation as well as robotics

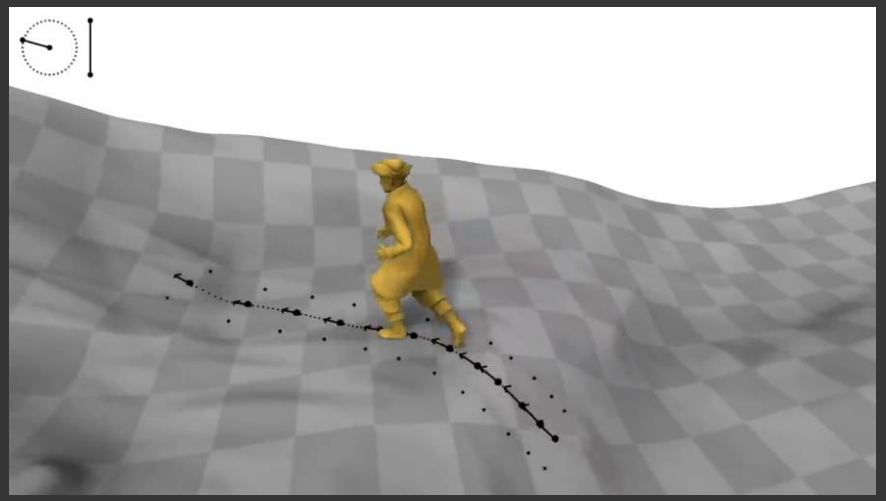
HRP2 robot



Oleksandr Stepanenko

Context

- Focus on Kinematics: branch of mechanics concerned with the motions of objects without regard to the forces that cause the motion
  - **Kinematics:** describes the positions of the body parts as a function of the joint angles
  - **Dynamics:** describes the positions of the body parts as a function of the body mass and applied forces.

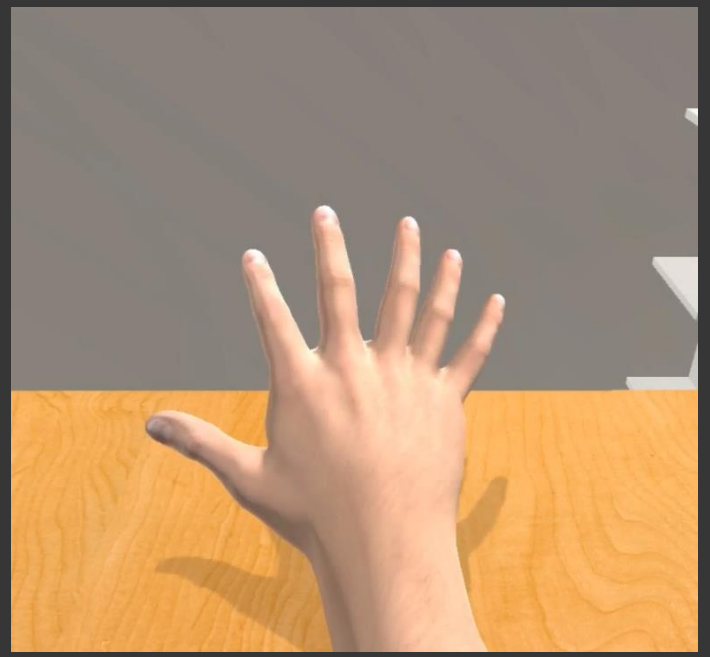


[Holden et al. 2018]

[Coros et al. 2010]

Who am I?

- Ludovic Hoyet, Inria Research Scientist, VirtUs team
  - Computer Graphics, Character Animation, Motion Capture, Perception



Course Map

- Focus on Motion Control

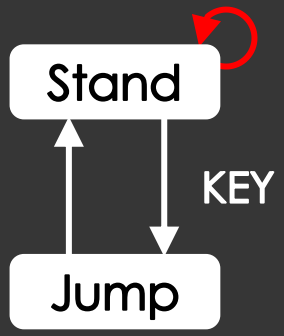
Complementary topics

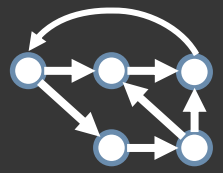
- Hierarchical Modeling
- Forward & Inverse Kinematics
- Motion Editing

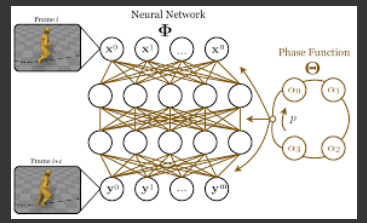
Motion Control

- Games and movies often require hundreds of animations
  - Unpractical to play them individually
  - Need some way to organise them
  - Need some way to intuitively control them
    - If possible in an online manner

Outline







2002 2015 2017

Finite State Machines

Motion Graphs

Motion Matching

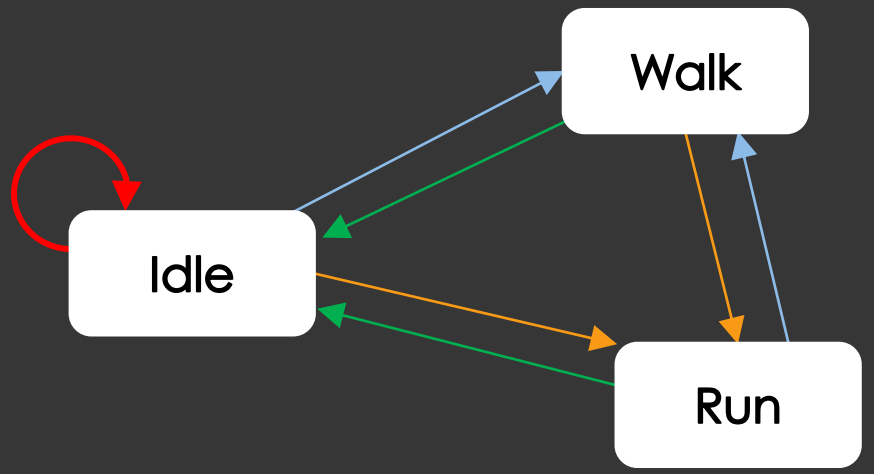
DL-based

- Other approaches exist(e.g., optimal control, physical controllers)
  - But focus on most commonly used in this presentation

State Machines

State Machines

- Actions of the characters are controlled in a state-driven manner



DO_WALK

DO_RUN

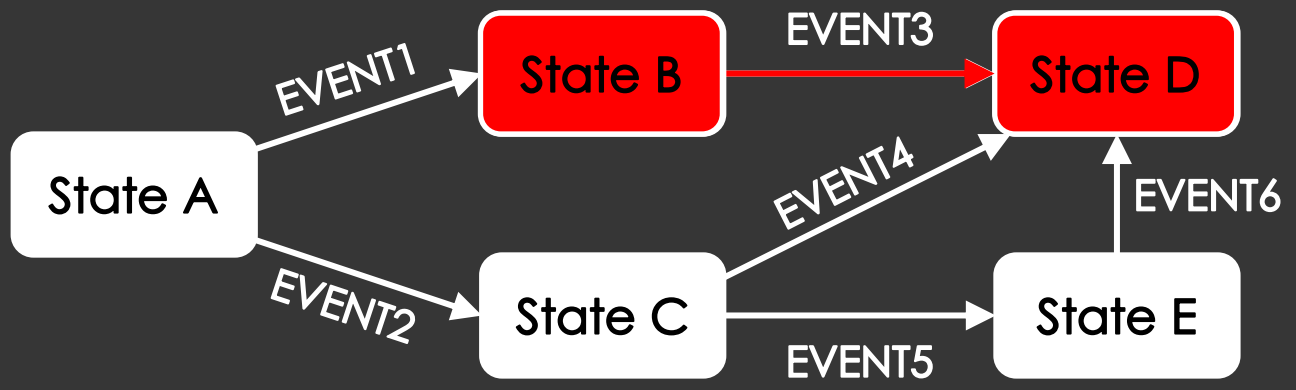
DO_STOP

→ Finite State Machine

– Responsible for smooth transitions

State Machines

- We will define a state machine as a connected graph of states and transitions
  - At any time, exactly one of the states is the current state
  - Transitions can happen instantaneously, or over time (blending)
  - Can be combined with motion editing (e.g., blending, IK)



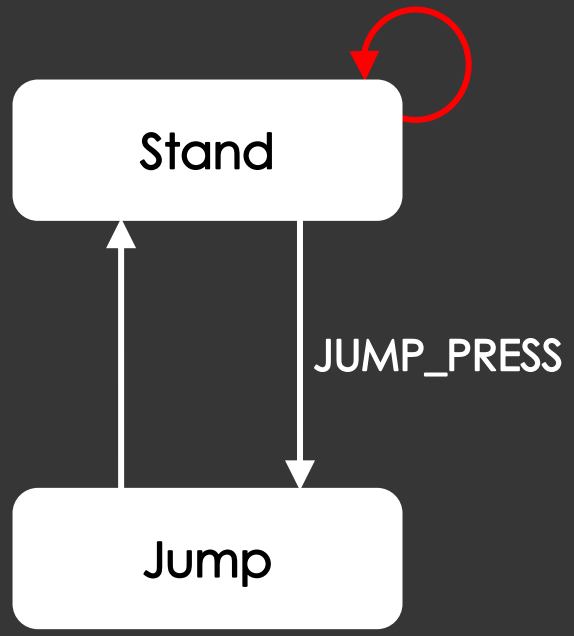
State Machines

- In the context of animation sequencing, we think of
  - States as representing individual animation clips
  - Transitions as being triggered by some sort of event

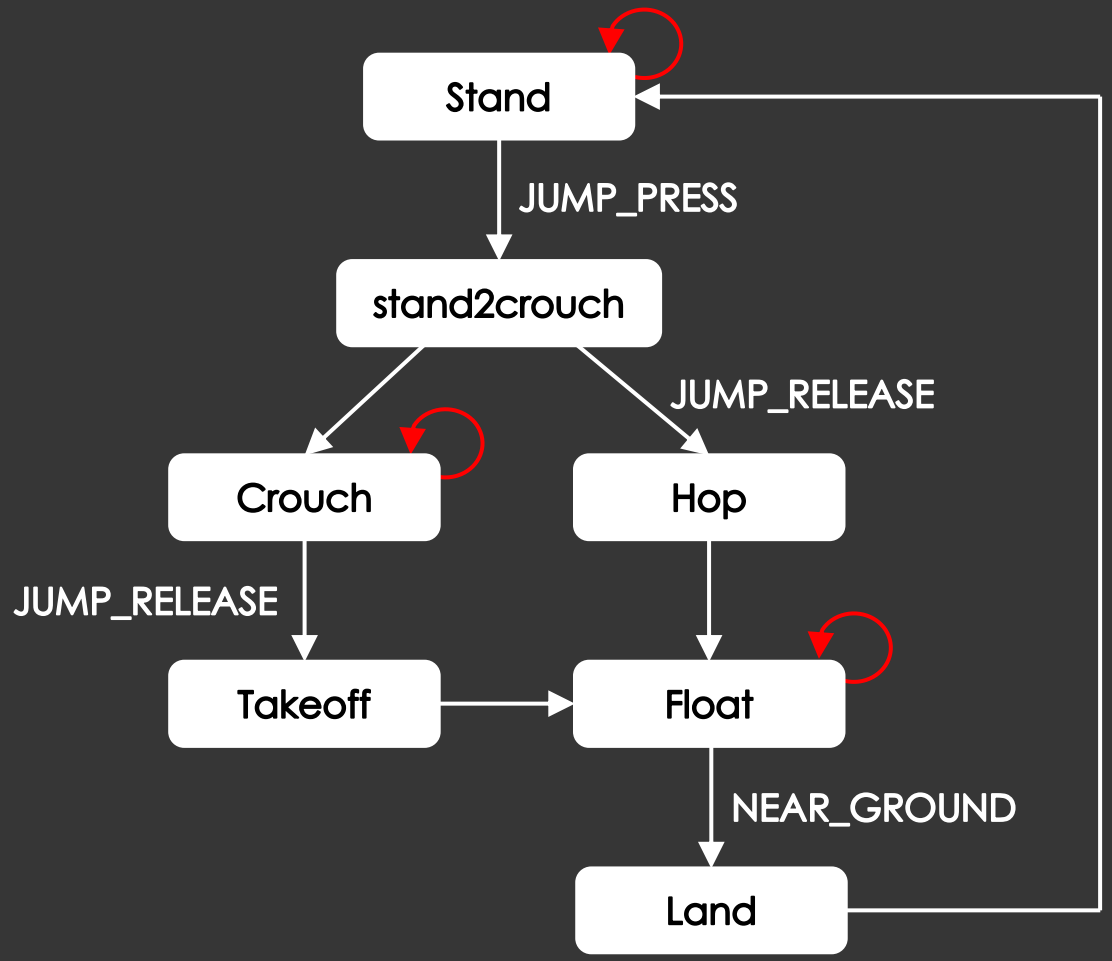
- An event might come from some internal logic (e.g., character getting close to a NPC) or some external input (e.g., button press)

Simple Jump State Machine

- Consider this simple state machine
  - Character jumps upon receiving a JUMP_PRESS message



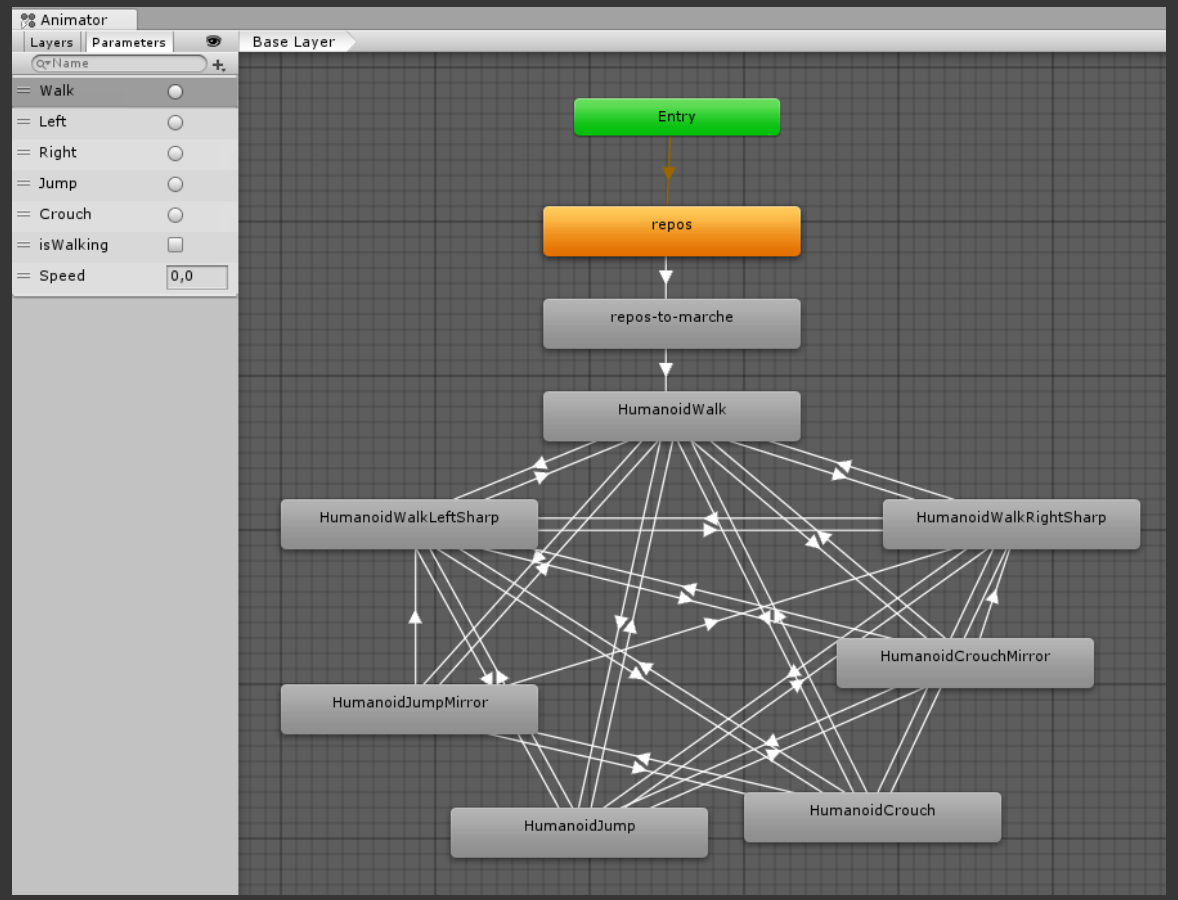
More Complex Jump



Creating State Machines

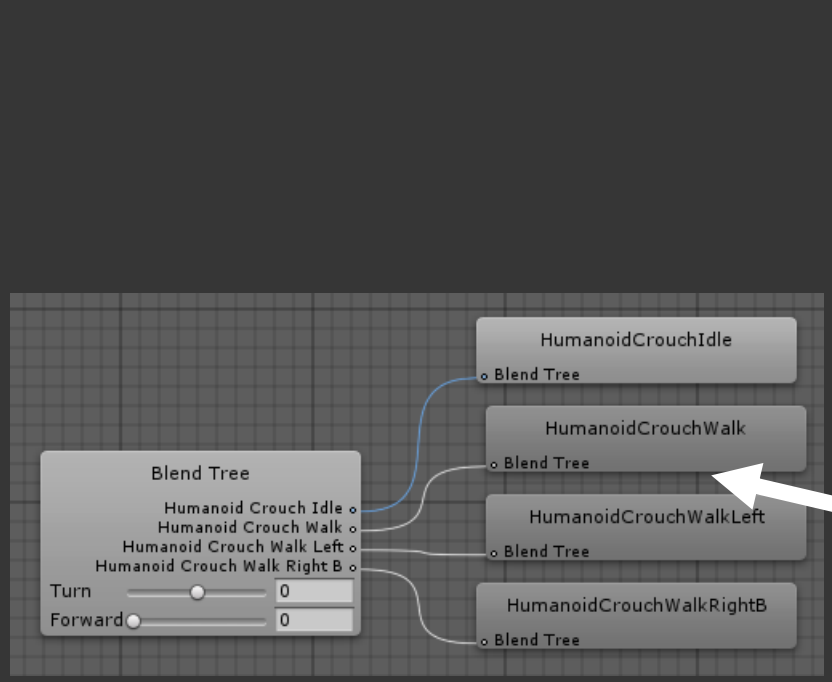
- Typing in text
- Graphical editor
- Automated generation (motion graphing)

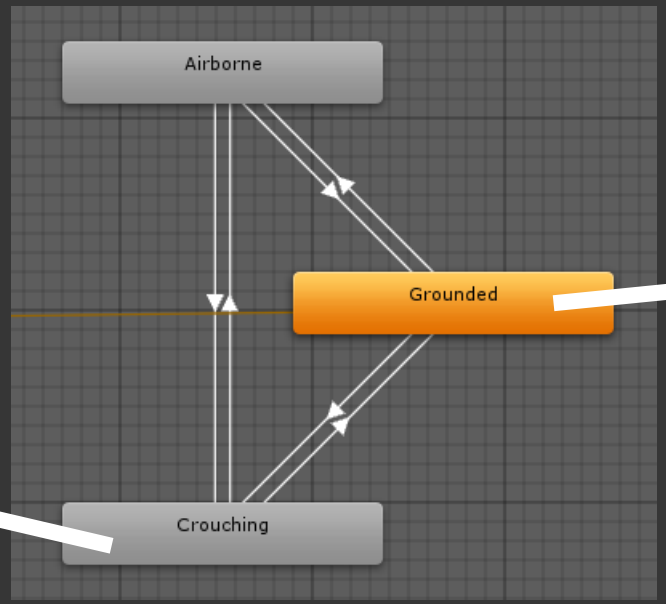
```
stand       {JUMP_PRESS          stand2crouch }
stand2crouch   {
            JUMP_RELEASE         hop
            END                  crouch
                        }
crouch      {JUMP_RELEASE        takeoff }
takeoff     {END                 float }
hop         {END                 float }
float       {NEAR_GROUND         land }
land        {END                 stand }
```

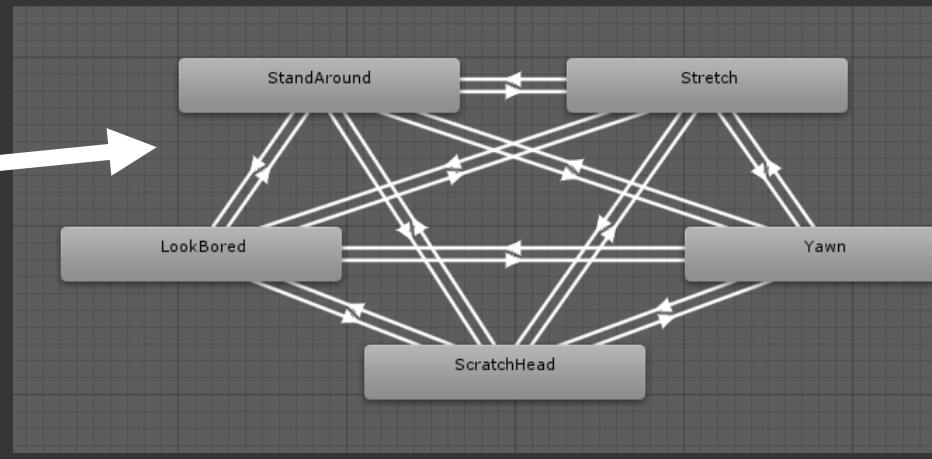


Nested State Machine

- We said that states usually represent individual animation clips
  - But it is actually also possible to combine State Machines together

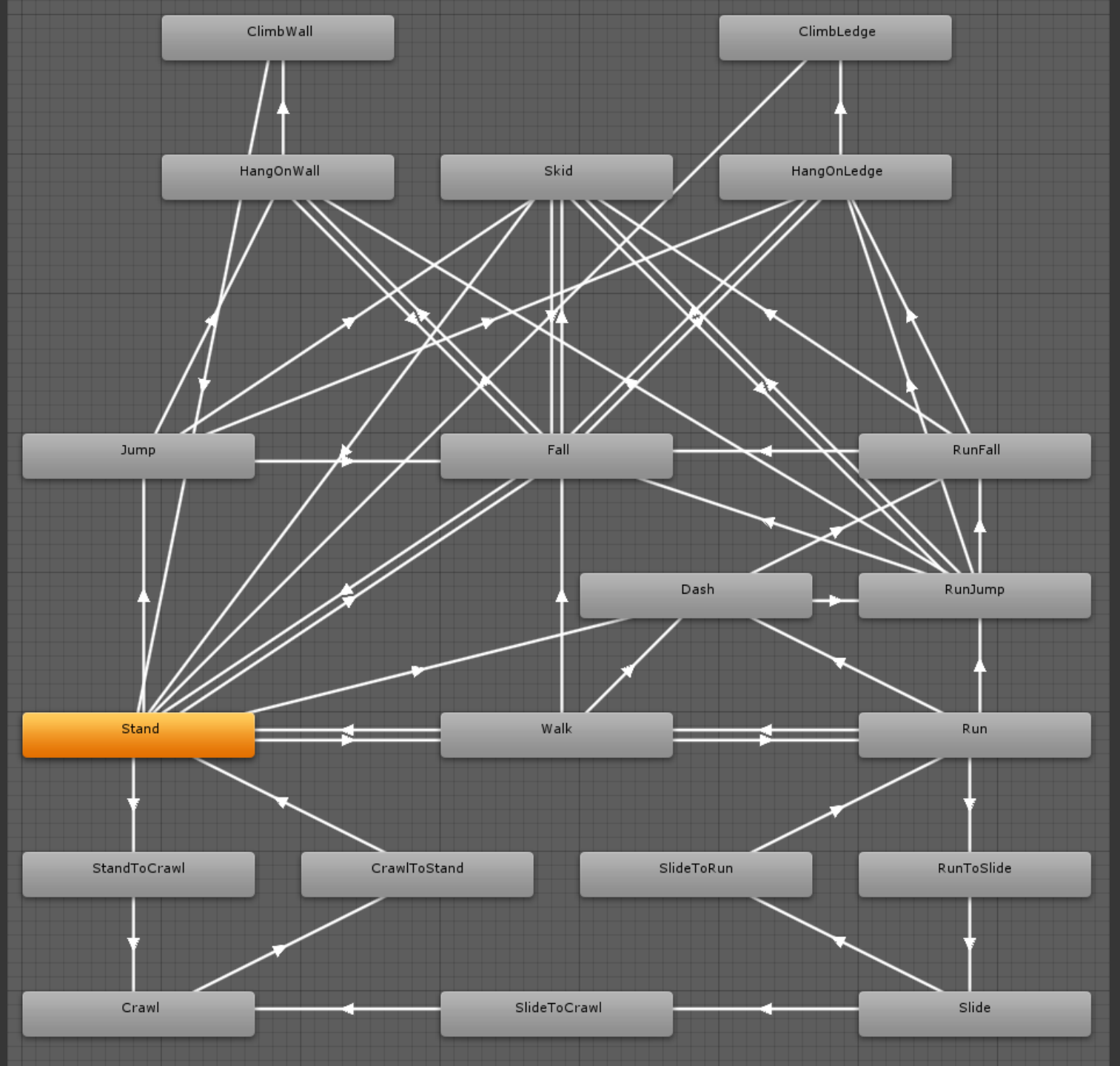




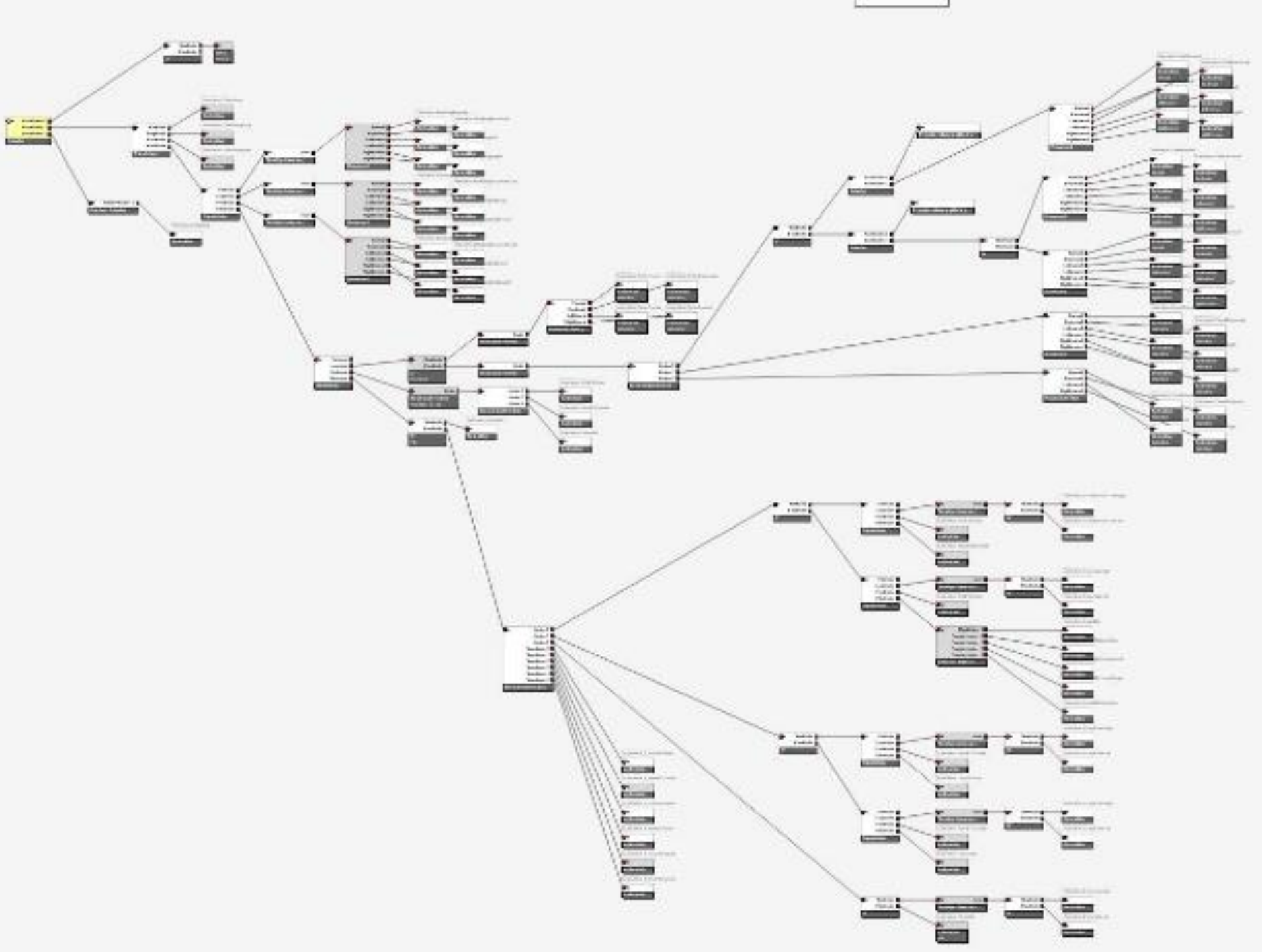


State Machines

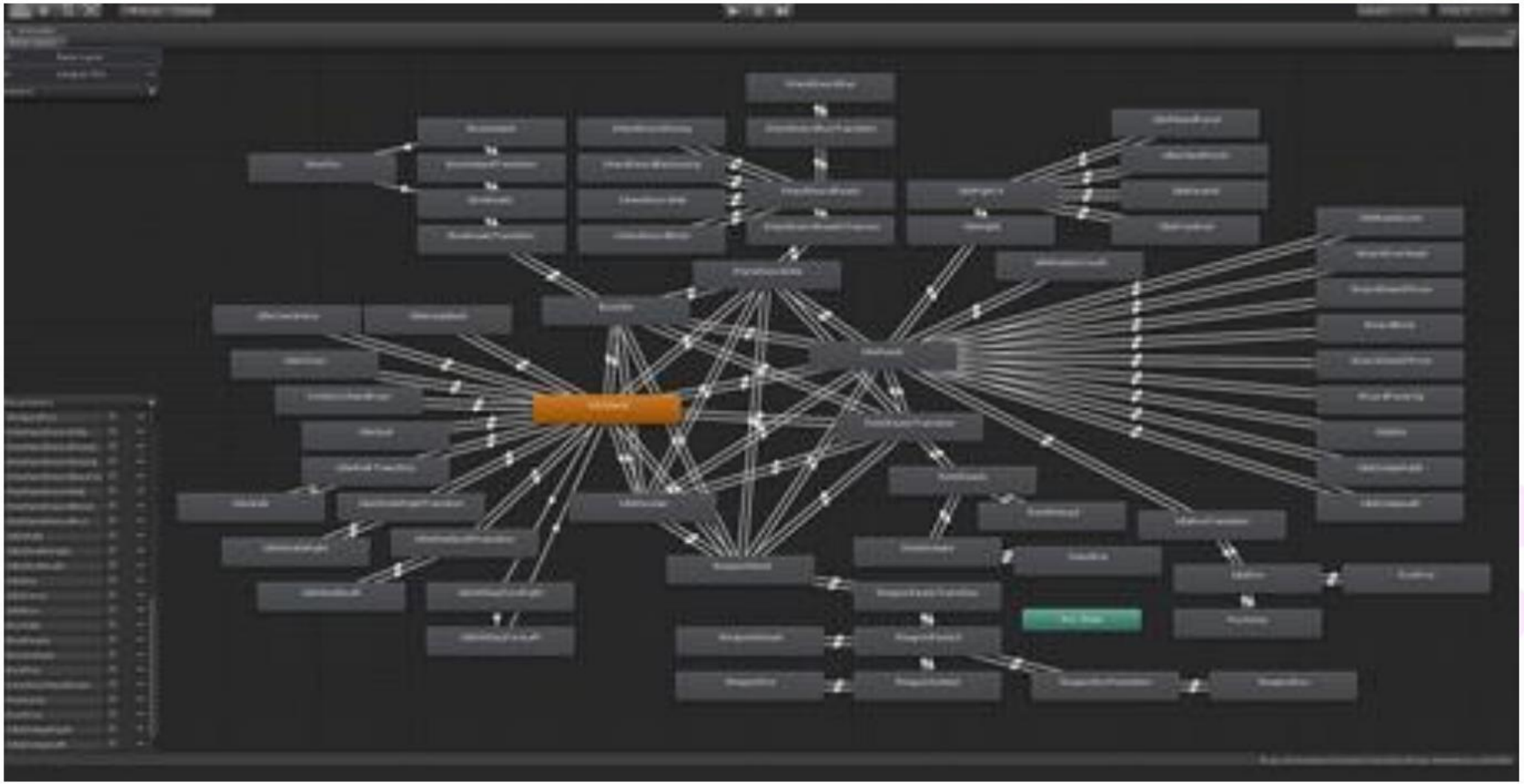
- Problems
  - Captured data can be voluminous
  - Processing motion data is intensive
  - Capturing all possible motion is impossible
  - Organising motions in state machines can be difficult



In "Motion Matching and The Road to Next-Gen Animation“, Simon Clavet, GDC 2016



Blend tree in
"Battlefield – Bad Company“
2008



State Machines

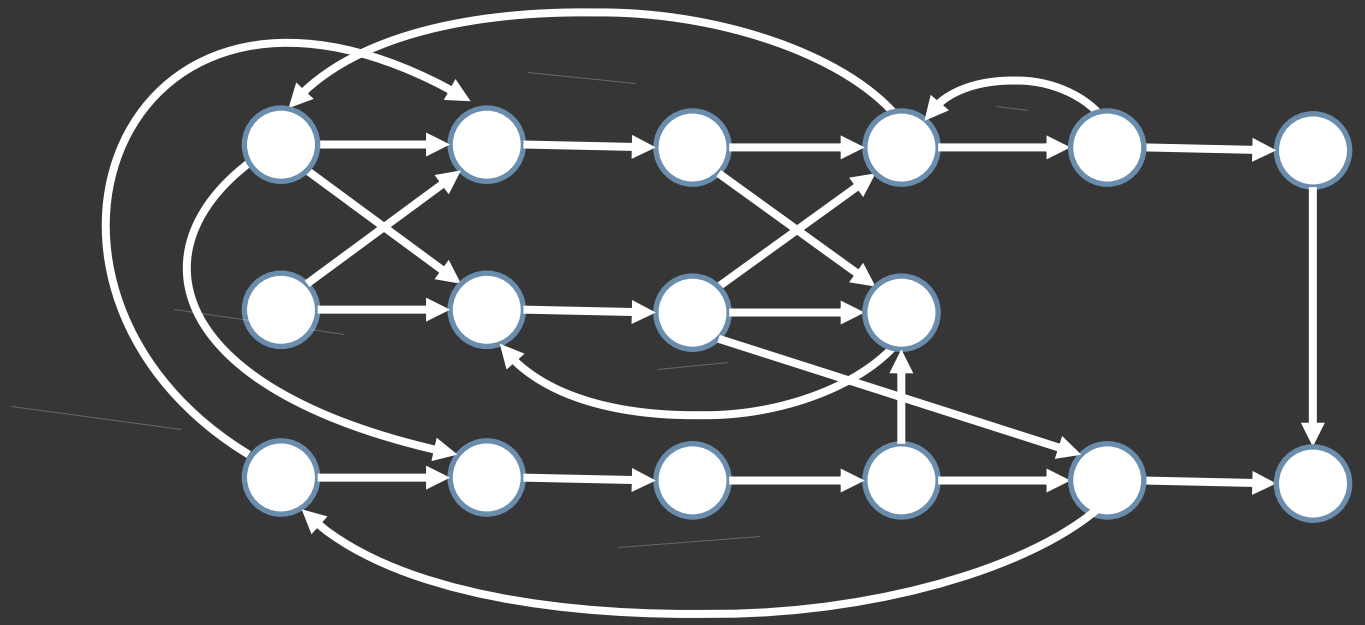
- Problems
  - Captured data can be voluminous
  - Processing motion data is intensive
  - Capturing all possible motion is impossible
  - Organising motions in state machines can be difficult

- Solution: Motion Graphs
  - Combine data intelligently to synthesize new motions

Motion Graph

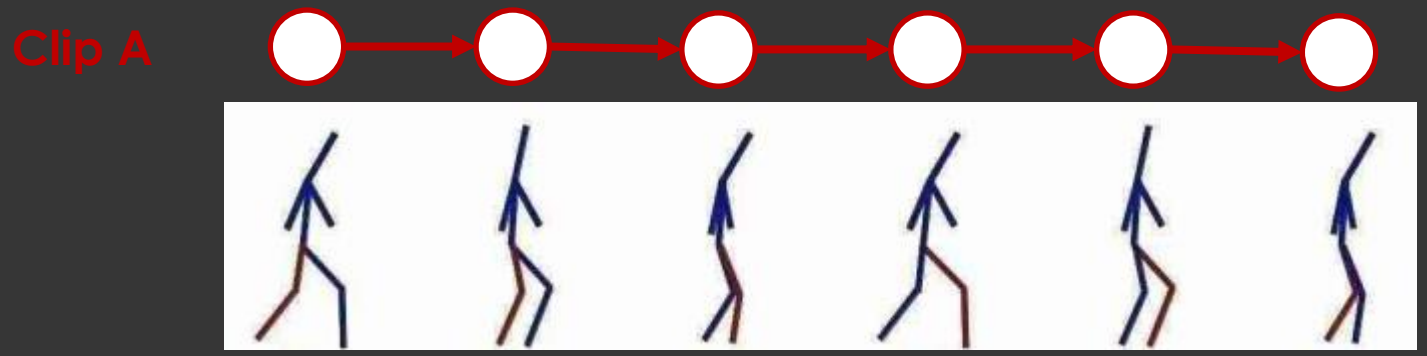
Motion Graph: Idea

- Goal: automate the organisation of motions clips and of potential transitions between frames
  - Introduced in 2002 by Kovar et al.: Motion Graphs



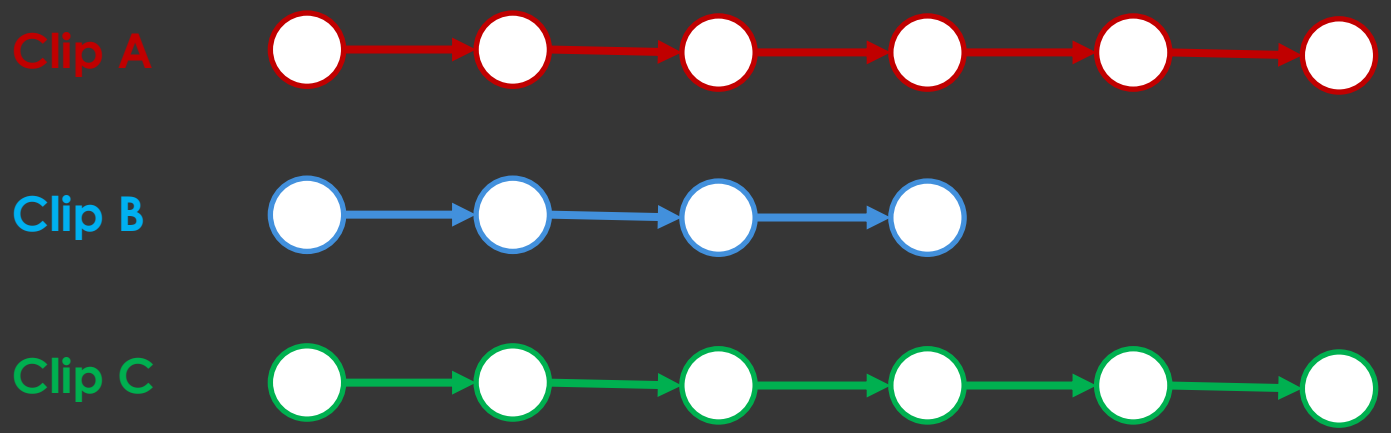
Motion Graph: Idea

- Every motion clip is a graph
  - Node: pose
  - Edge: transition between two successive frames



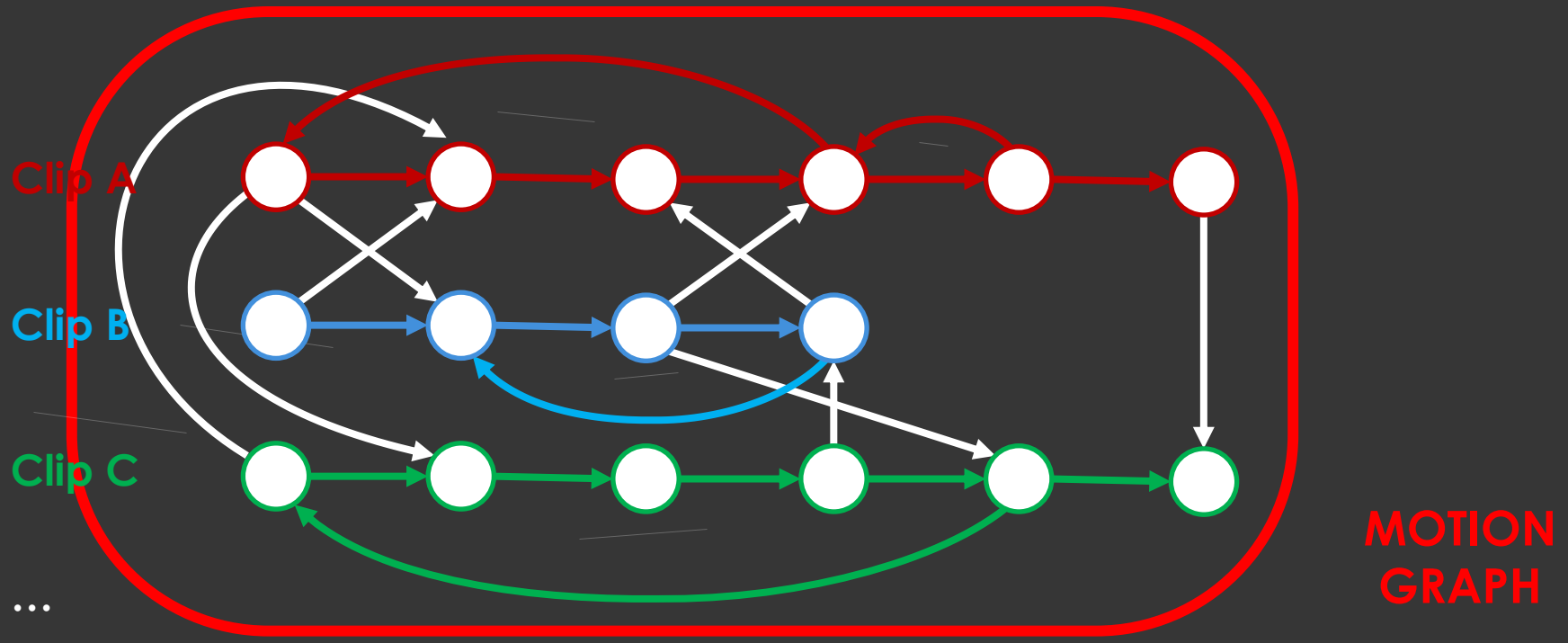
Motion Graph: Idea

- There are many such clips in a motion database



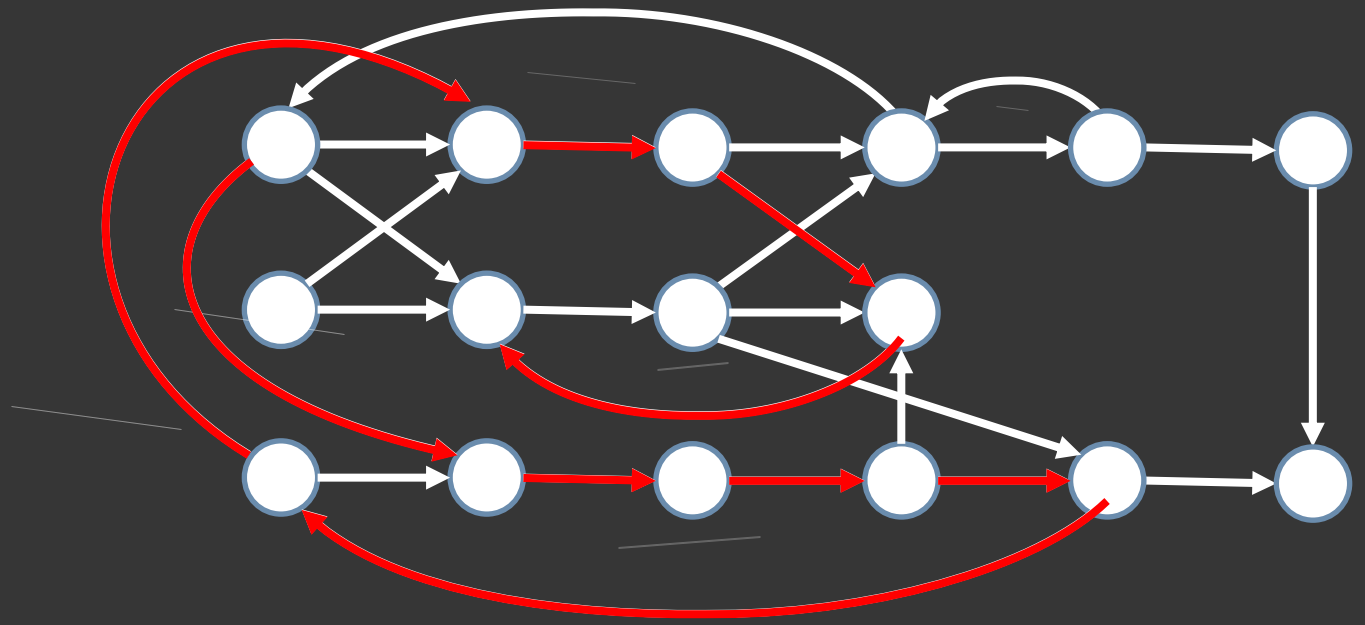
Motion Graph: Idea

- There are many such clips in a motion database
- Find similar poses within and between clips
  - Add transitions between them



Motion Graph: Idea

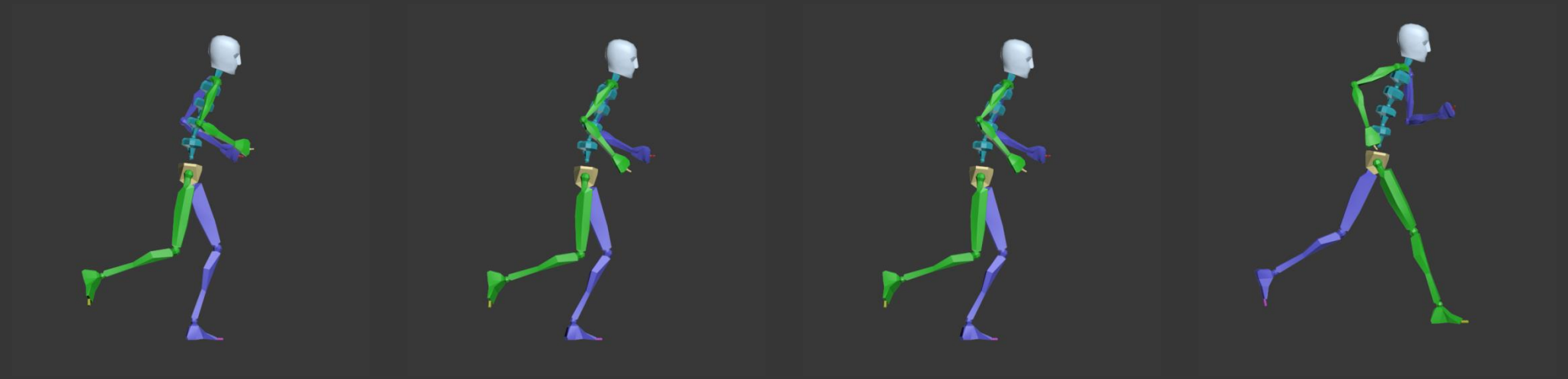
- Now any walk on this graph generates a new, smooth motion



Construction

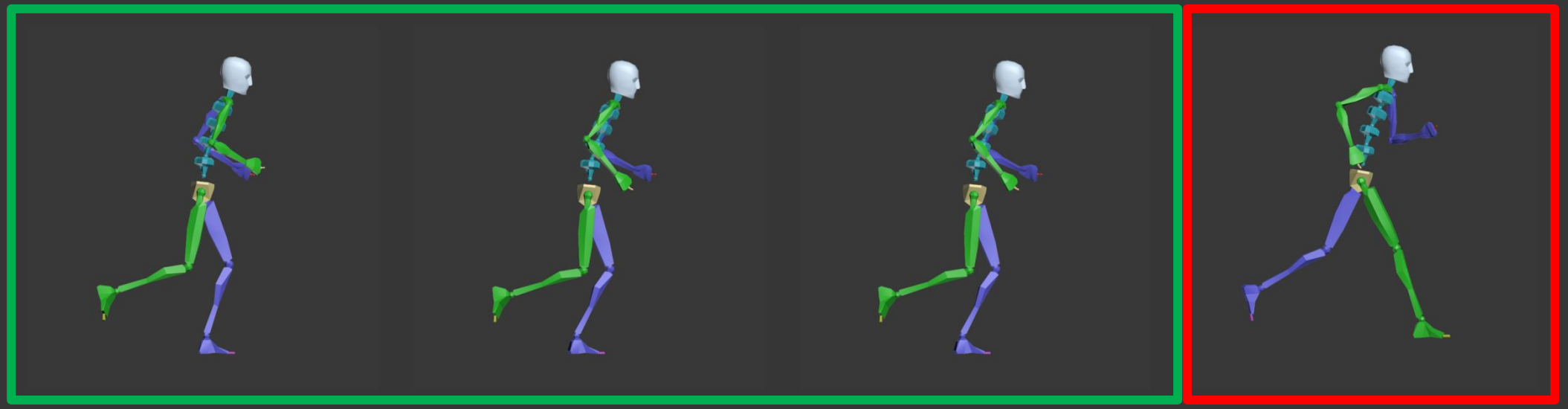
- Measure similarity between poses across clips

Where are motions similar enough to move from one to the other with minimal disruption?



Pose Similarity

- Simplest: joint angle differences
  - Relative importance (shoulder vs. wrist)
  - e.g. $\Sigma\, w(j)*(\theta(j_s) - \theta(j_t))^2$



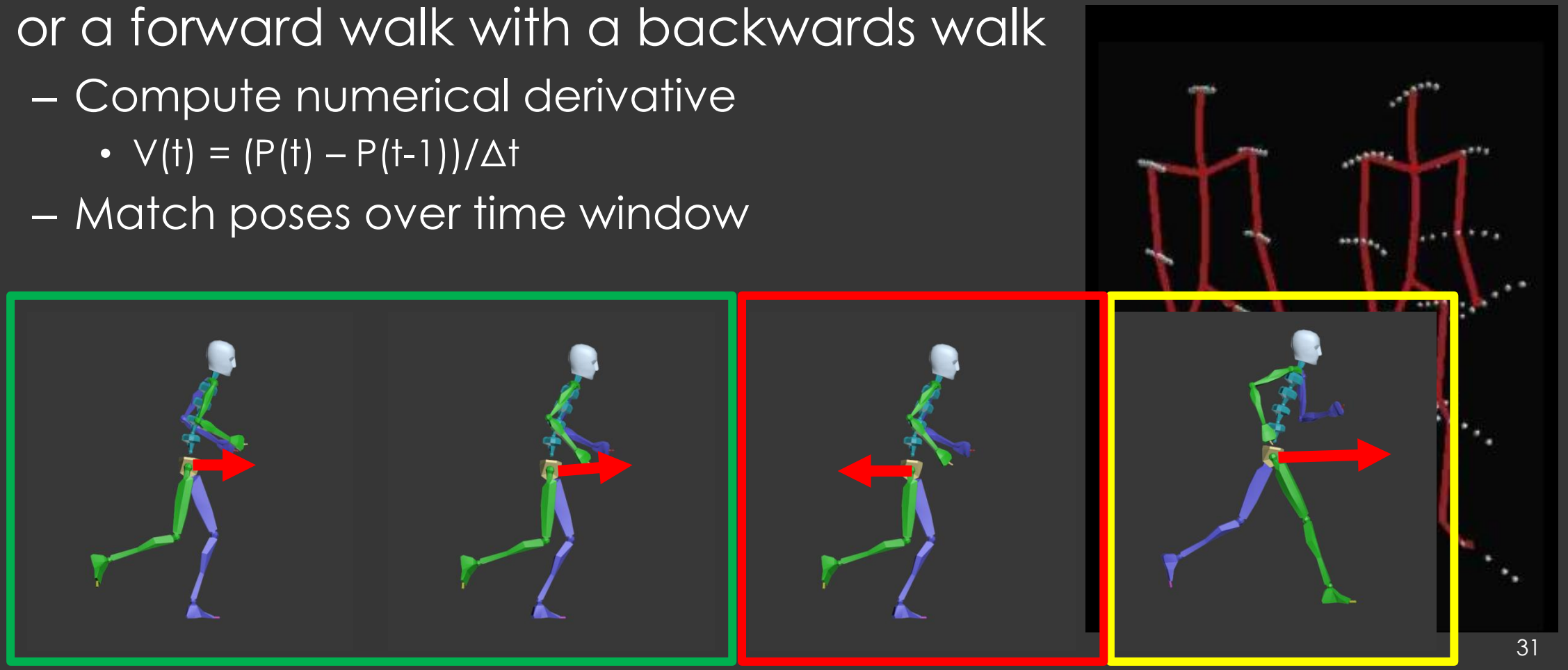
Pose Similarity

- Simplest: joint angle differences
  - Relative importance (shoulder vs. wrist)
  - e.g. $\Sigma\, w(j)*(\theta(j_s) - \theta(j_t))^2$

- Alternatives
  - Alignment of virtual points on body
  - Visual difference of projection
  - Perceptual model (perception-based per-joint similarity weights)

Velocity Similarity

- To avoid splicing a touch and a punch, or a forward walk with a backwards walk
  - Compute numerical derivative
    - $V(t) = (P(t) - P(t-1))/\Delta t$
  - Match poses over time window



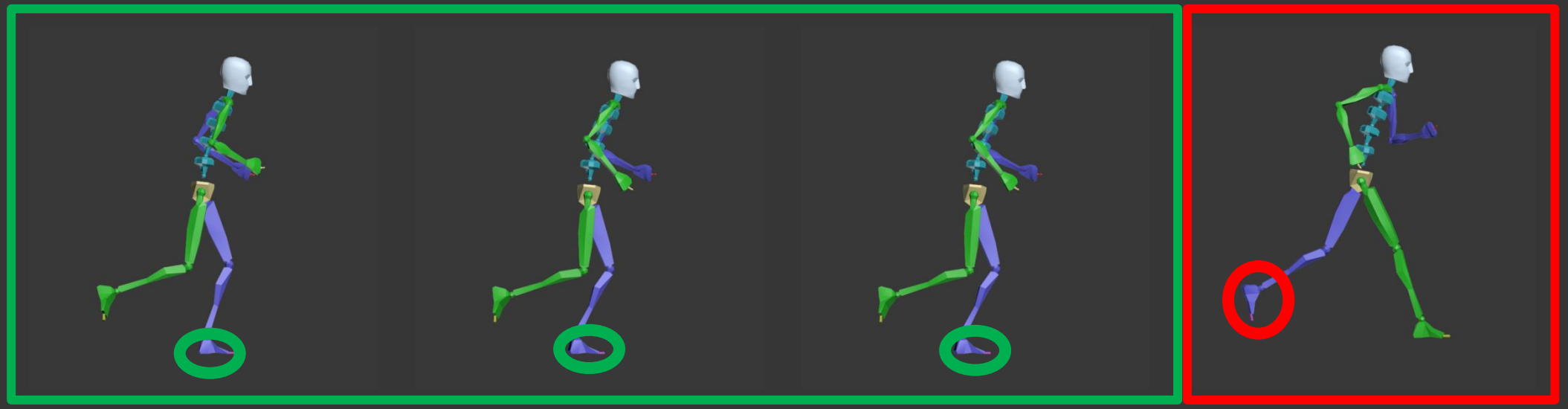
Interaction Similarity

- Contact with the environment should remain crisp
  - Smoothing often loses this
  - e.g. feet sliding on/in ground

- Pose/velocity measures enforce this poorly

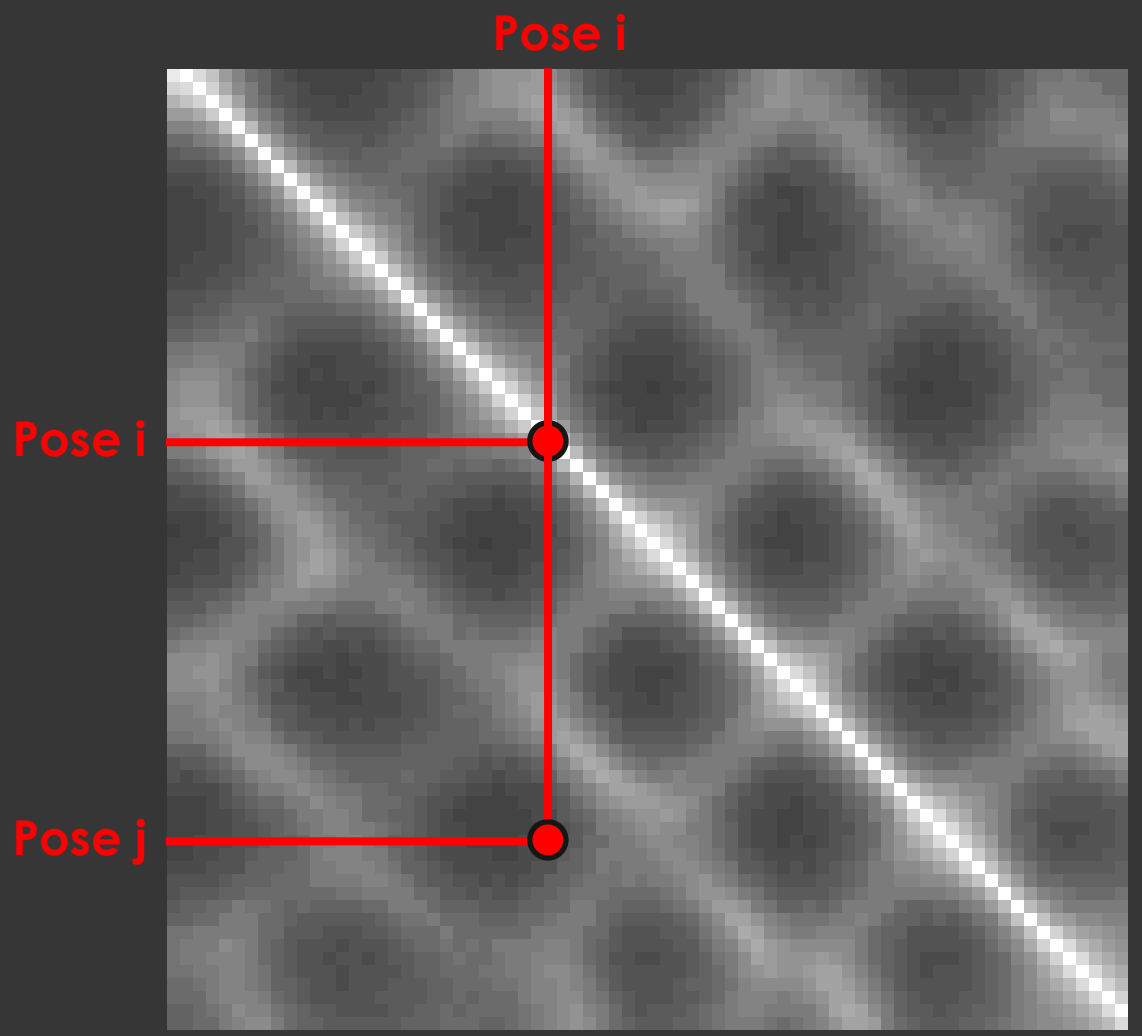
Interaction Similarity

- May want to penalize score if motions differ in interaction with environment
  - e.g., setting foot down vs. standing still



Similarity Matrices

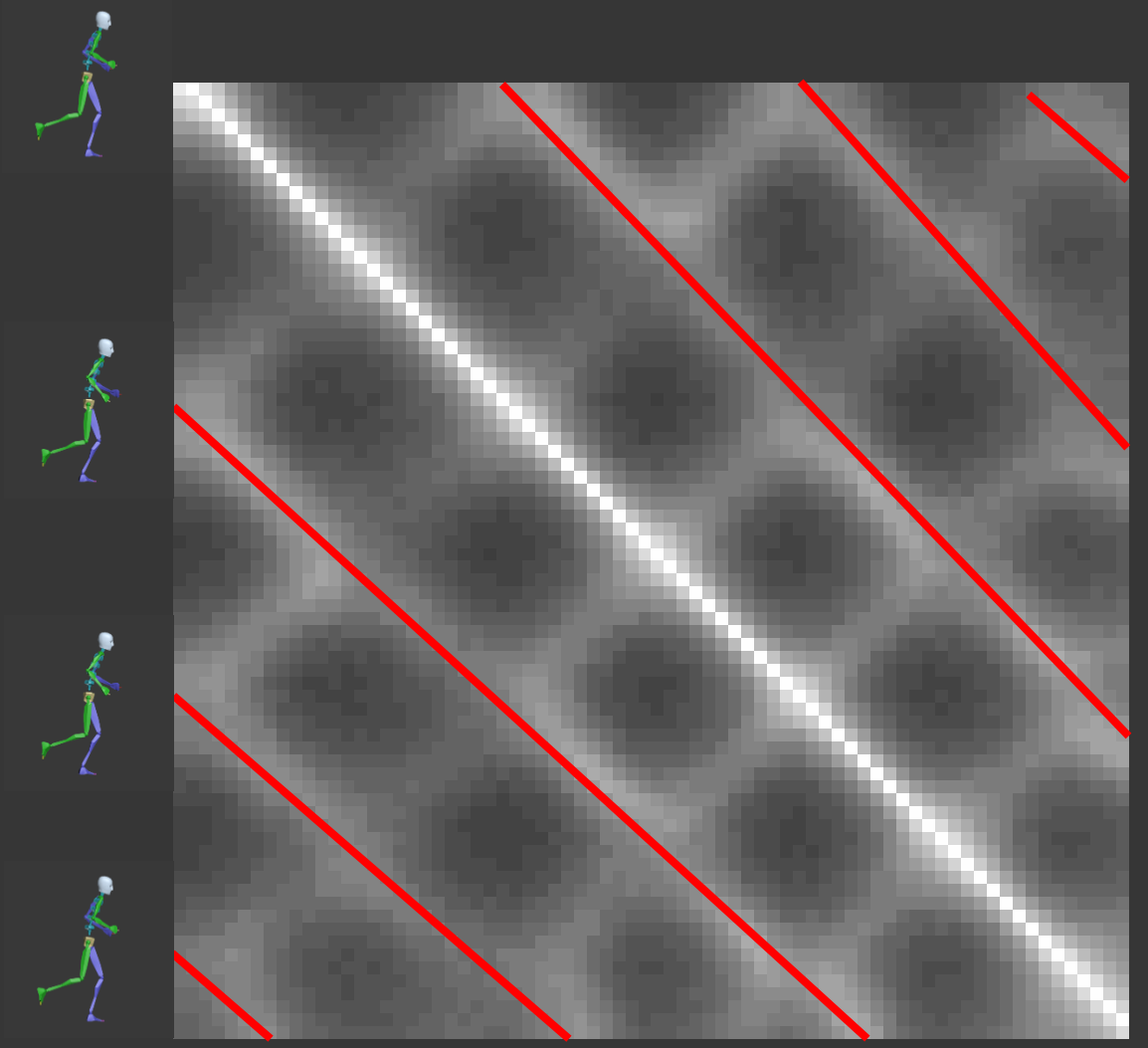
- Similarities can be computed between each pair of frames of different motions
  - Provides information about how similar frame are
- Can be visualized as a similarity matrix



Running vs. Running

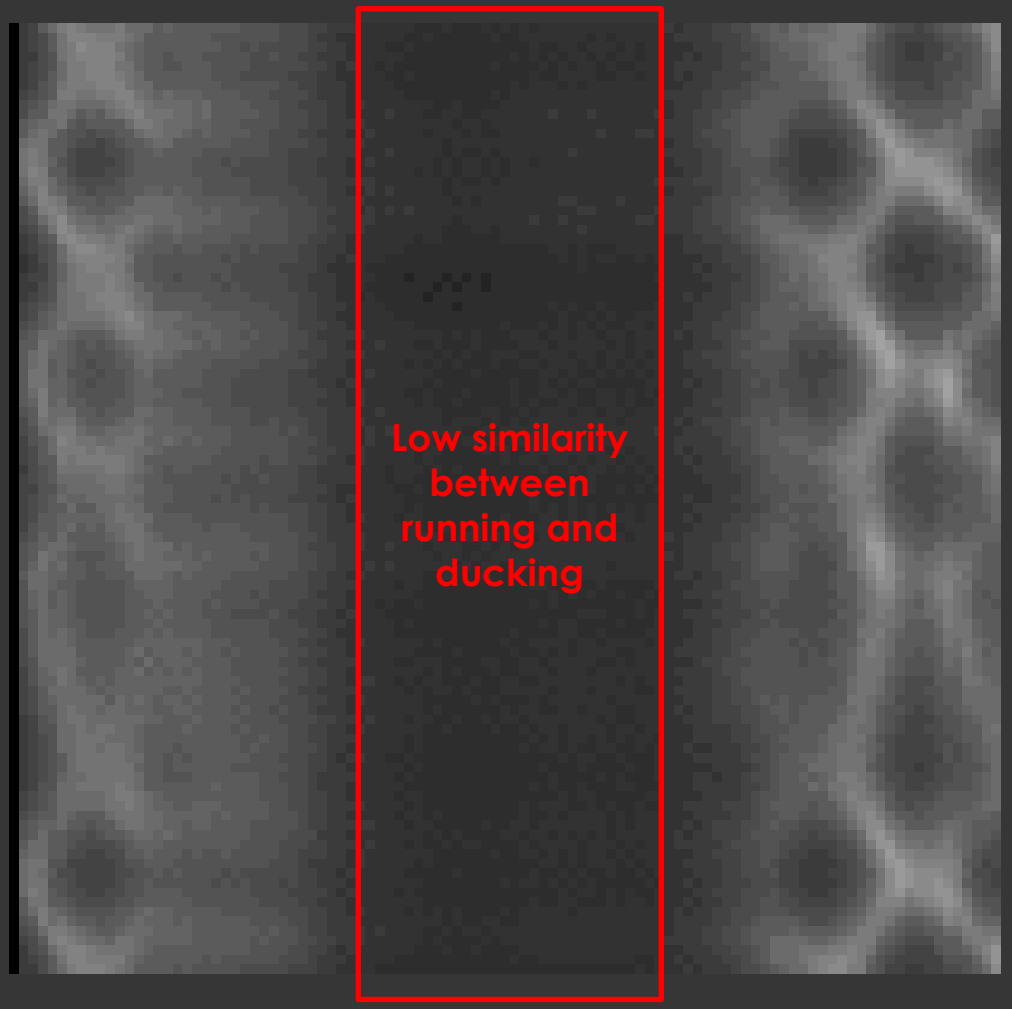
Similarity Matrices

- Similarities can be computed between each pair of frames of different motions
  - Provides information about how similar frame are
- Can be visualized as a similarity matrix



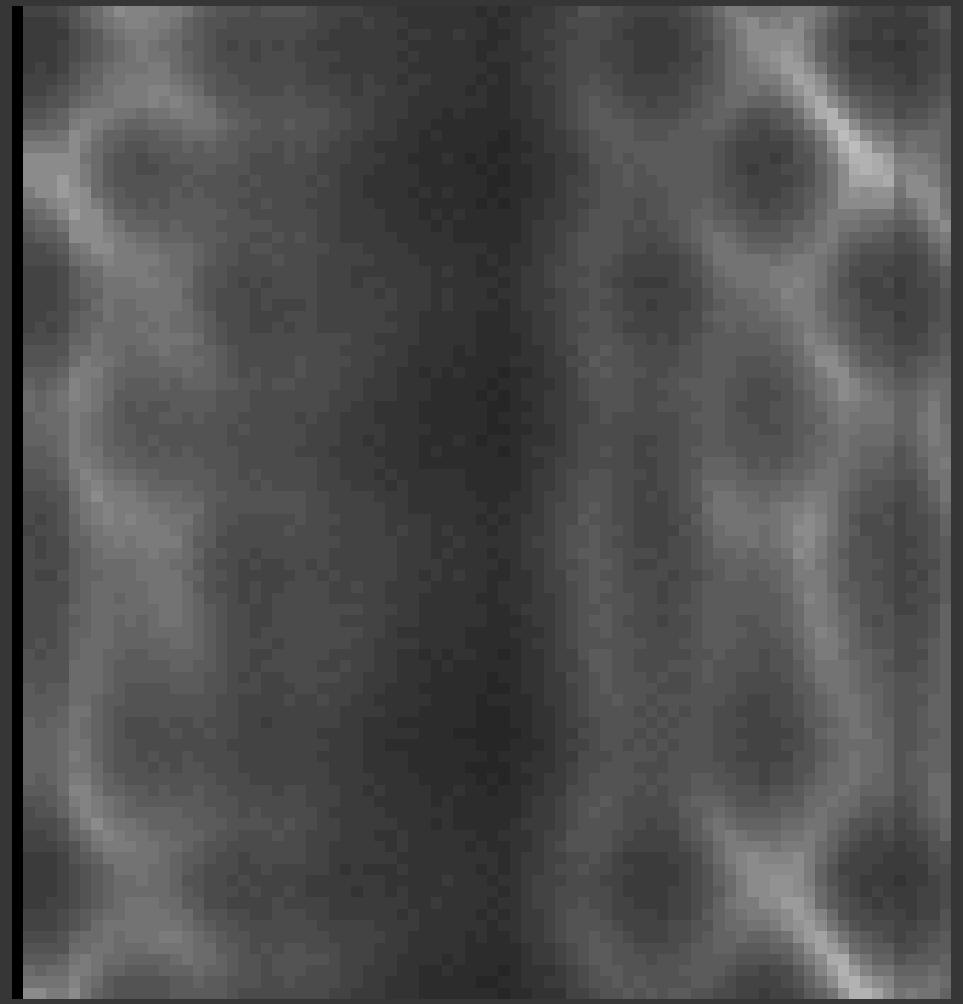
Running vs. Running

Similarity Matrices



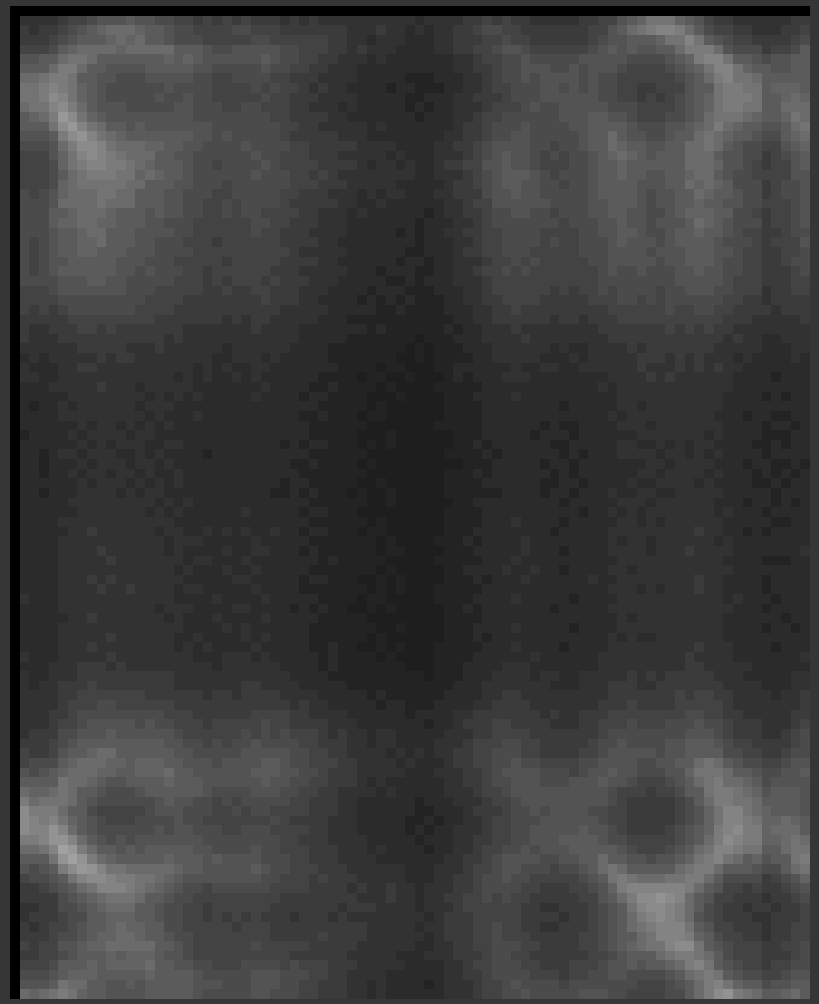
Running vs. Ducking

Similarity Matrices



Running vs. Punching

Similarity Matrices



Ducking vs. Punching

Example Motion Graph

- Source motions:
  - Run:
  - Duck:
  - Punch:

Step 1: Build Similarity Matrices

```
For each frame i in MOTION_A
      For each frame j in MOTION_B
            cost(i,j) =   w(pose) * posecost(i,j)
                        + w(vel)  * velocitycost(i,j)
                        + w(acc)  * accelcost(i,j)
            similarity(i,j) = 1/cost(i,j)
```

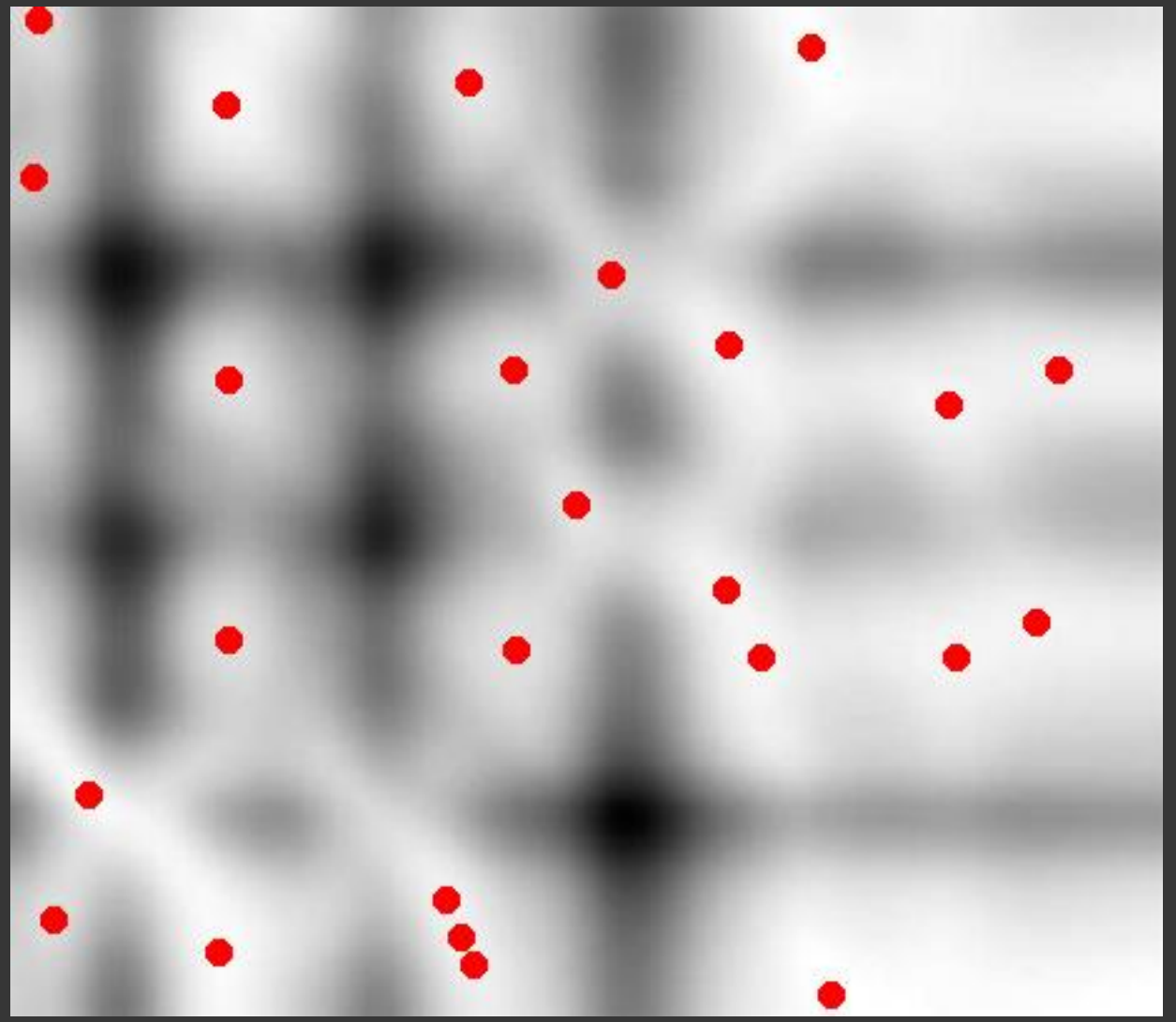
With for instance

```
      posecost(i,j)     = Σ w(joint)*|joint(i)-joint(j)|
      velocitycost(i,j) = Σ w(joint)*|jointvel(i)-jointvel(j)|
      accelcost(i,j)    = Σ w(joint)*|jointacc(i)-jointacc(j)|
```

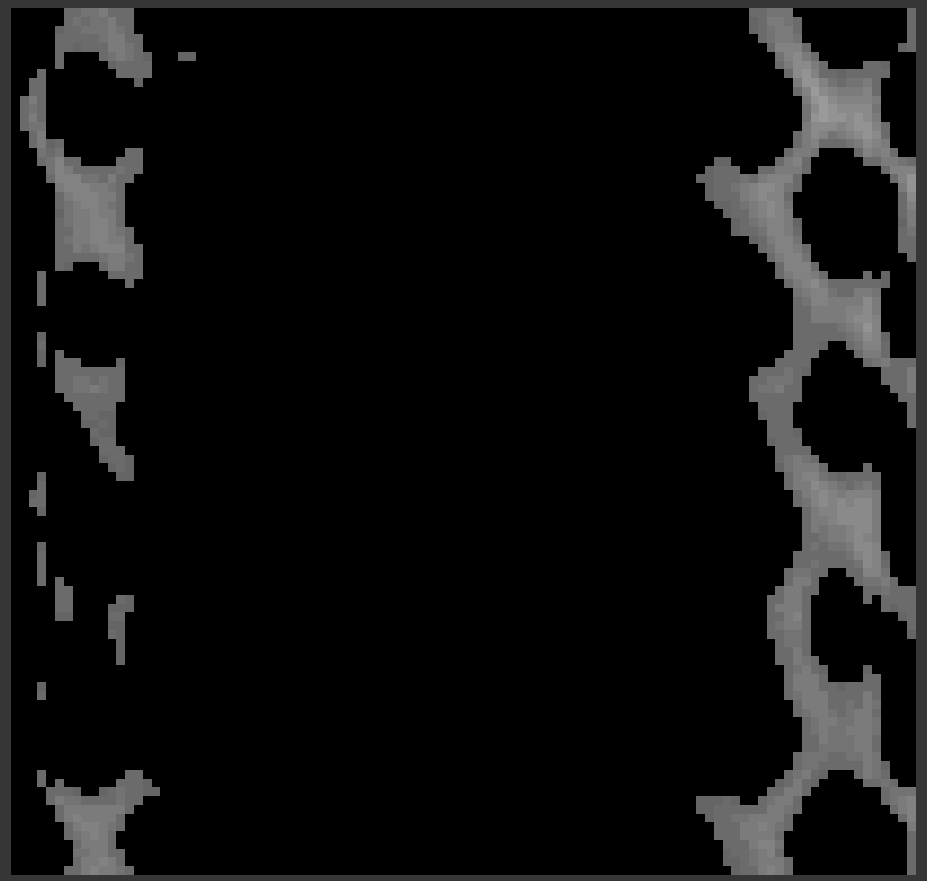
Also: `if MOTION_A = MOTION_B: cost(i,j) = cost(j,i)`

Step 2: Identify Transition Candidates

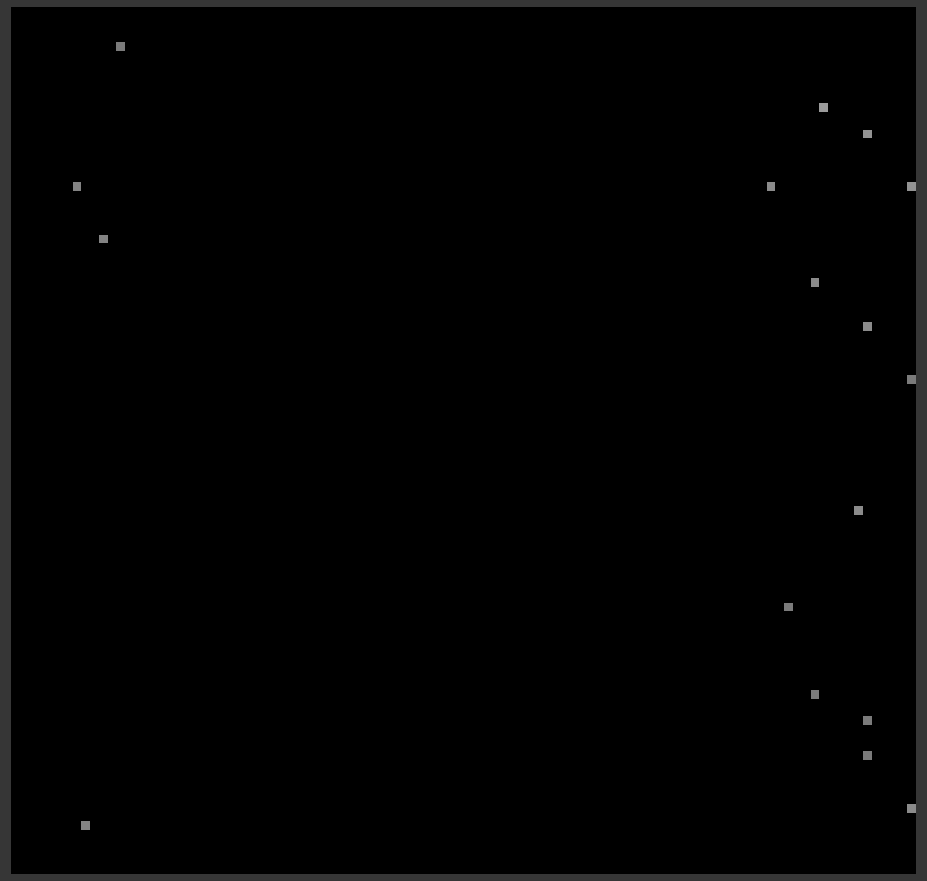
- High-quality motions tend to cluster
  - Near-transitivity
- Want only a representative of each cluster
  - Remove values which are not local maxima



Step 2: Identify Transition Candidates



Threshold 0.7

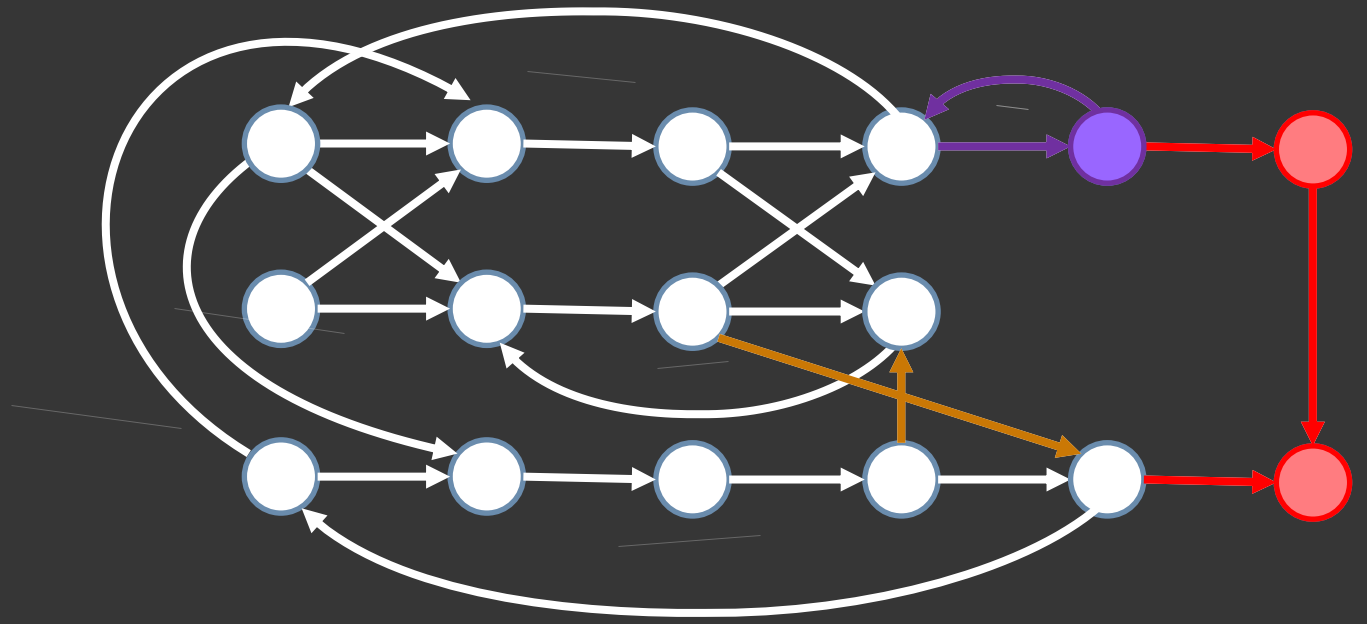


Local Maxima

Running vs. Ducking

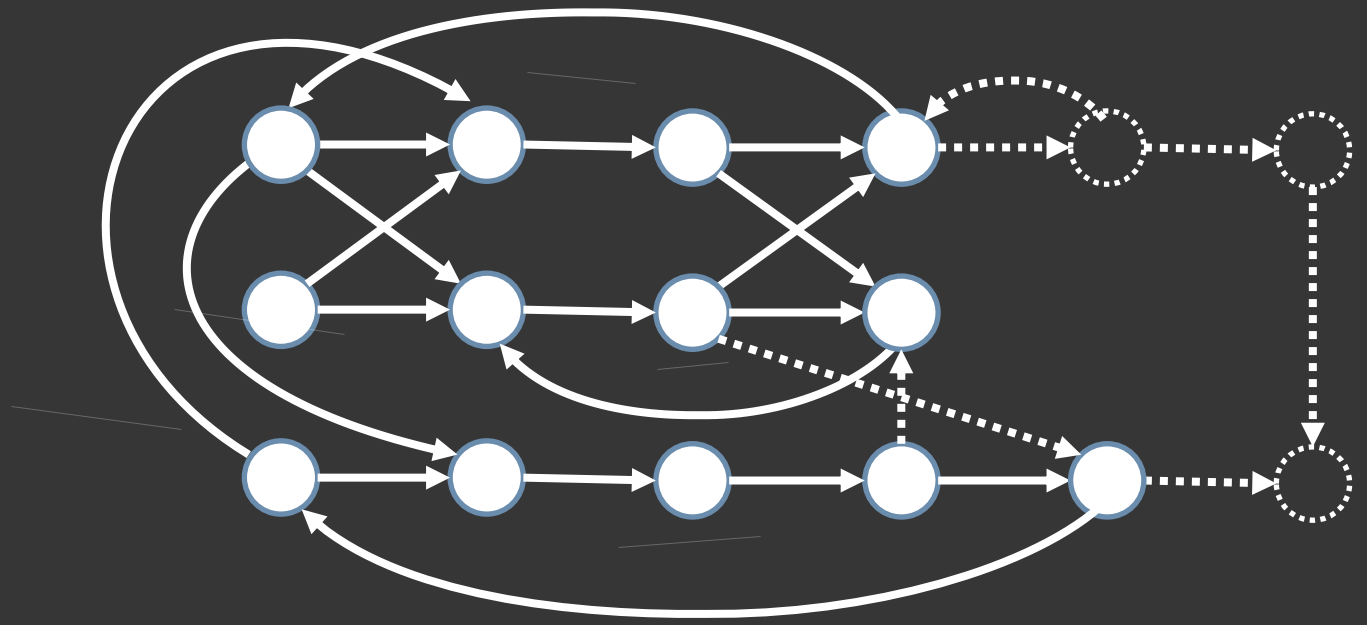
Step 3: Pruning Edges

- Problems
  - Dead ends / Holes (infinite loops)
  - Semantic discontinuities (e.g., dancing to boxing)
  - Sinks (~loops with less than N nodes)



Step 3: Pruning Edges

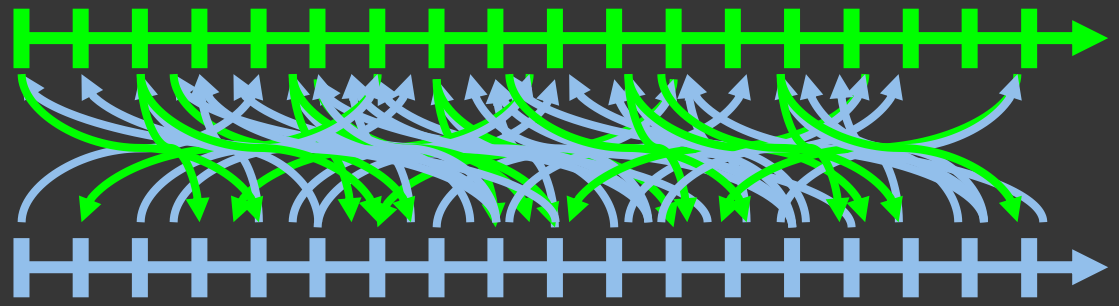
- Solutions
  1. Compute the biggest Strongly Connected Component
  2. Remove semantic inconsistencies (e.g., label motions)



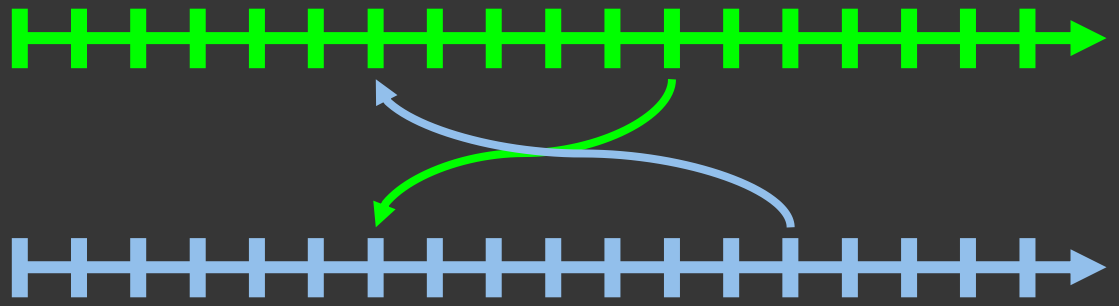
Basic Motion Graph

- Acquire motion
- Create similarity matrices
- Cull matrices to local maxima, threshold
- Build graph
- De-seam transitions at runtime

Basic Motion Graph



Low threshold:
too many transitions



High threshold:
too few transitions

- Problem: hard to get a specific result
  - Optimizable
  - Or just tweak it. A lot.

Searching for a motion

- Random walks on the motion graph are not interesting
  - So we search for motion that satisfies some objective

- Goal: find a complete graph walk w that minimizes f

$$f(w) = f([e_1, \dots, e_n]) = \sum_{I=1}^{n} g([e_1, \dots, e_{i-1}], e_i)$$

  - g should give some guidance throughout the entire motion, as arbitrary motion is not desirable
    - Minimum distance, minimum traveling time, pose accuracy…

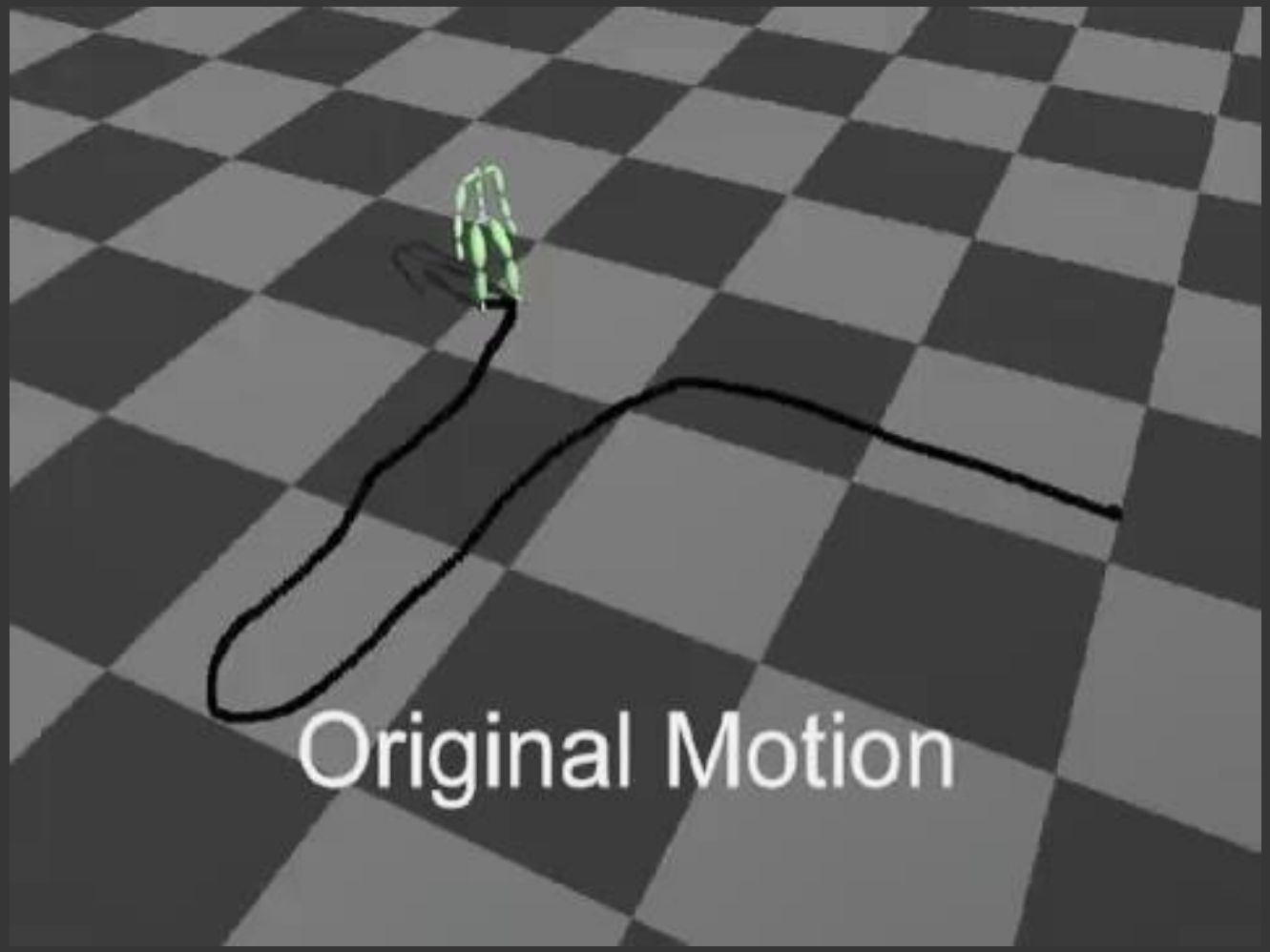
Searching for a motion

- Branch and bound strategy: cull branches of the search incapable of yielding a minimum

Path Synthesis

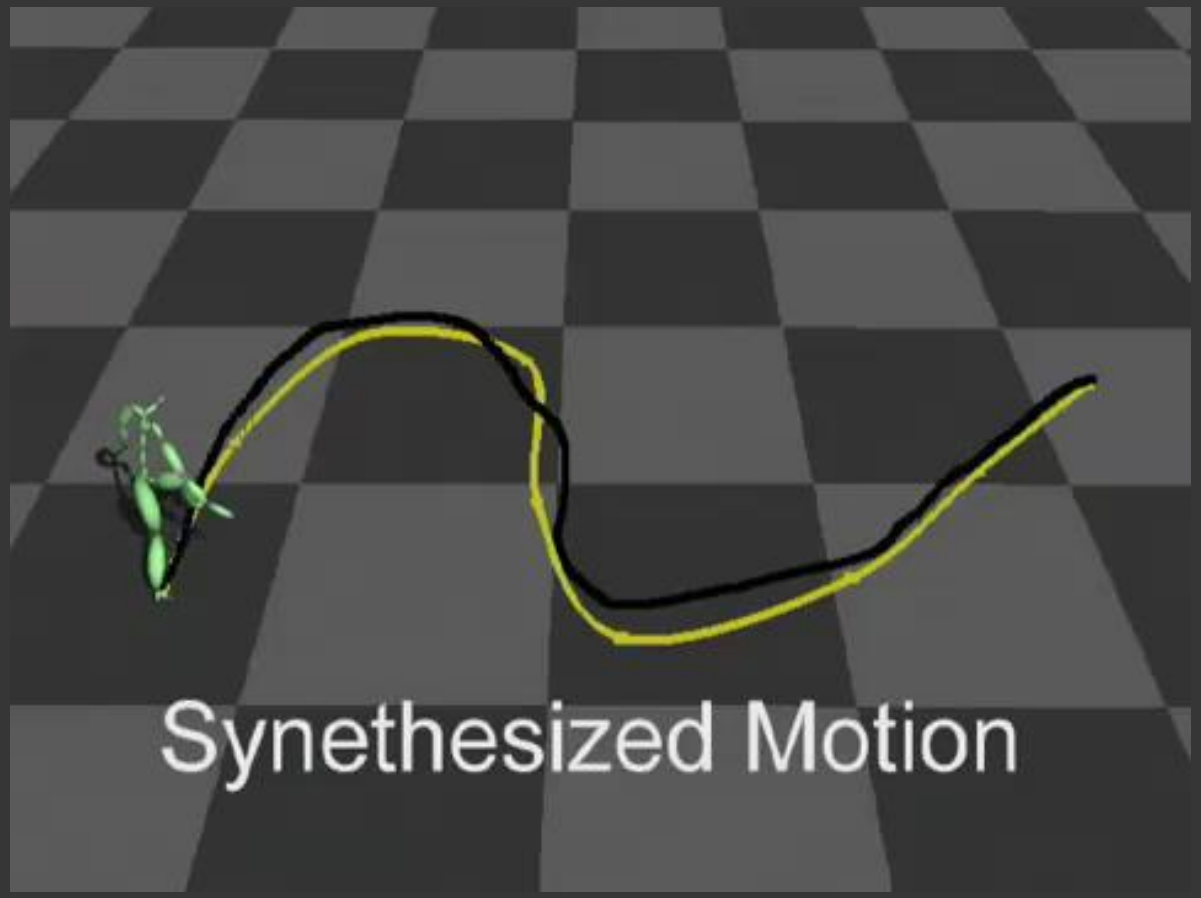
- Project root onto the floor at each frame, forming a piecewise linear curve
  - Minimise the distance between optimal and actual path created by sequence of motion clips
  - Confine the search to the subgraph containing appropriately labelled motion

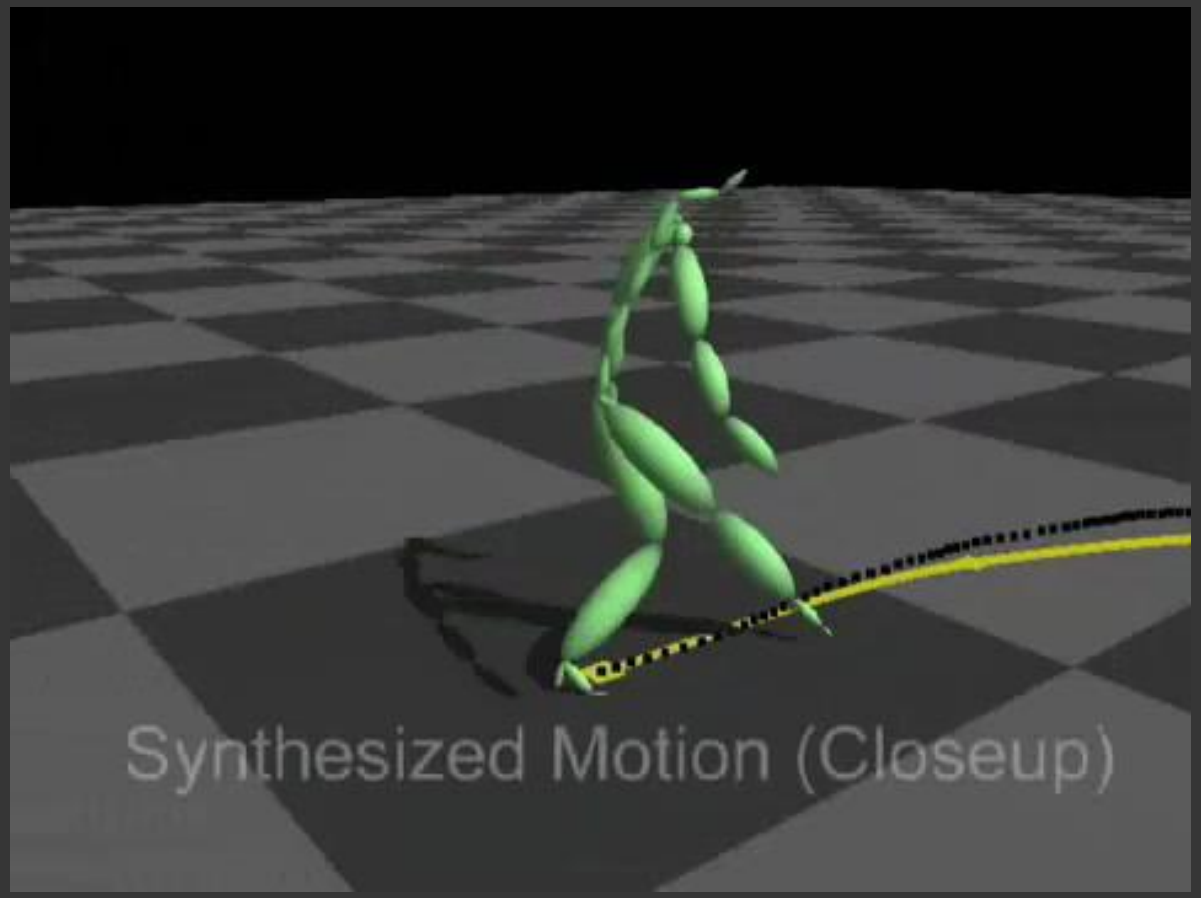
Path Synthesis



Videos: https://research.cs.wisc.edu/graphics/Gallery/kovar.vol/MoGraphs/

Path Synthesis

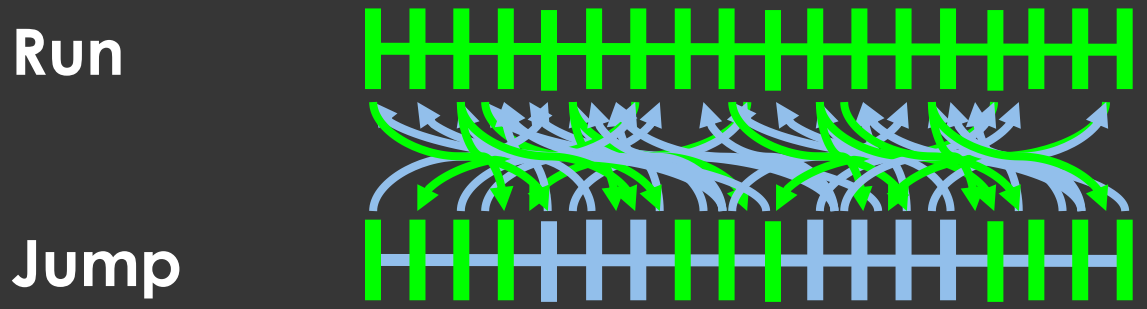




Videos: https://research.cs.wisc.edu/graphics/Gallery/kovar.vol/MoGraphs/

Other Issues

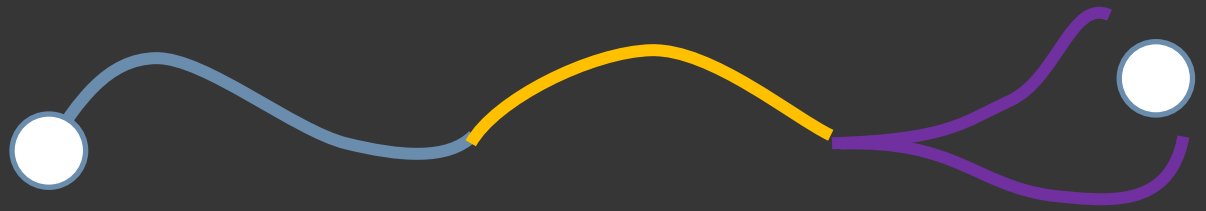
- Suppose you have a motion graph...



...with plenty of transitions.

Other Issues

- Available source motions might not perform the task
  → Lack of motion capability



Other Issues

- Available source motions might not perform the task
  → Lack of motion capability
- Solutions
  – Warp resulting motion to respect the final trajectory (how much acceptable?)

Other Issues

- Available source motions might not perform the task
  → Lack of motion capability
- Solutions
  - Warp resulting motion to respect the final trajectory (how much acceptable?)
  - Interpolate graph walks (Safonova et al. Construction and optimal search of interpolated motion graph)

Motion Graph

Advantages

- Re-use of high-quality input
  - Minimal editing, so maintain characteristics
- Automatic
  - Large amounts of data
  - Novice users
  - Natural transitions between behaviours
  - As-good-as-possible added transitions
- Fast and easy to use at runtime

Disadvantages

- Require balance between number/quality of transitions
- Can be difficult to get a specific result / control

→ Hardly used in video games

Motion Matching

Motion Matching

- "*a ridiculously brute-force approach to animation selection*" done by "*looking at all motion capture data and jumping at the best place, every frame*". (Clavet, 2016)

- Spawns from three questions:
  - "Where would you put this animation in your structure?"
  - "Is there a way to deal with loops and transitions in a uniform way?"
  - "How do we choose the next animation?"

Motion Matching: road to next-gen animation. Simon Clavet, Ubisoft Montreal, 2016.
https://www.gdcvault.com/play/1023280/Motion-Matching-and-The-Road

Motion Matching

- Motion Graphs (and similar approaches) compute the cost of a transition offline, and decimate the graph

- Motion Matching computes this cost at runtime, look at **ALL** available mocap
  - Huge memory footprint, which would have made old consoles and PC struggle → would have affected other elements of pipeline

General Idea

- Many variations/implementations of the MM algorithm

- General idea:
  - Every N frames: search database of animation for a frame best matching some set of features
  - Features: describe the current context + desired user properties

  → If a frame with a lower cost than the current frame is found, a transition is inserted, and the new animation is blended in

Feature Vector

- Describes the features we wish to match at each frame
  - Different feature vectors can describe many different behaviours
  - E.g., for locomotion controller $x = \{t^t\ t^d\ f^t\ \dot{f}^t\ \dot{h}^t\} \in R^{27}$ [Holden et al. 2020]
    - $t^t \in R^6$: 2D future trajectory positions
    - $t^d \in R^6$: 2D future trajectory facing directions
    - f^t and $\dot{f}^t \in R^6$: two foot joint positions/velocities
    - $\dot{h}^t \in R^3$: hip joint velocity

Projected on the ground
(20, 40, 60 frames in the future at 60Hz)

Local to the character

Pose Vector

- Describes all the pose information for a single frame of animation
  - e.g., pose information $y = \{y^t\ y^r\ \dot{y}^t\ \dot{y}^r\ \dot{r}^t\ \dot{r}^r\ o^*\}$ [Holden et al. 2020]
    - $y^t\ y^r$: joint local translations and rotations
    - $\dot{y}^t\ \dot{y}^r$: joint local translational and rotational velocities
    - $\dot{r}^t\ \dot{r}^r$: character root translational and rotational velocity (local to the character forward facing direction)
    - o^*: other additional task specific outputs (e.g., foot contact, position of other characters, future positions of some joints)

Matching and Animation Databases

- Feature and pose vectors are computed for each frame and concatenated into large matrices
  - Matching database
  - Animation database

Runtime

- Every N frames, or when user input changes significantly:
  - Construct query vector $\hat{x}$ (desired feature vector)
    - Pose-based features are extracted from the entry in the Matching Database corresponding to the current frame i
    - Other user controls are constructed via other means, e.g., estimated future trajectory position/direction
  - Then find entry k in the Matching Database which minimizes $||\hat{x} - xk||^2$
    - If k != i → transition in the database, and blended
  - To limit search to specific motion gaits/tags: ranges in the Animation and Matching Databases can be pre-computed and the search limited to these ranges

Runtime

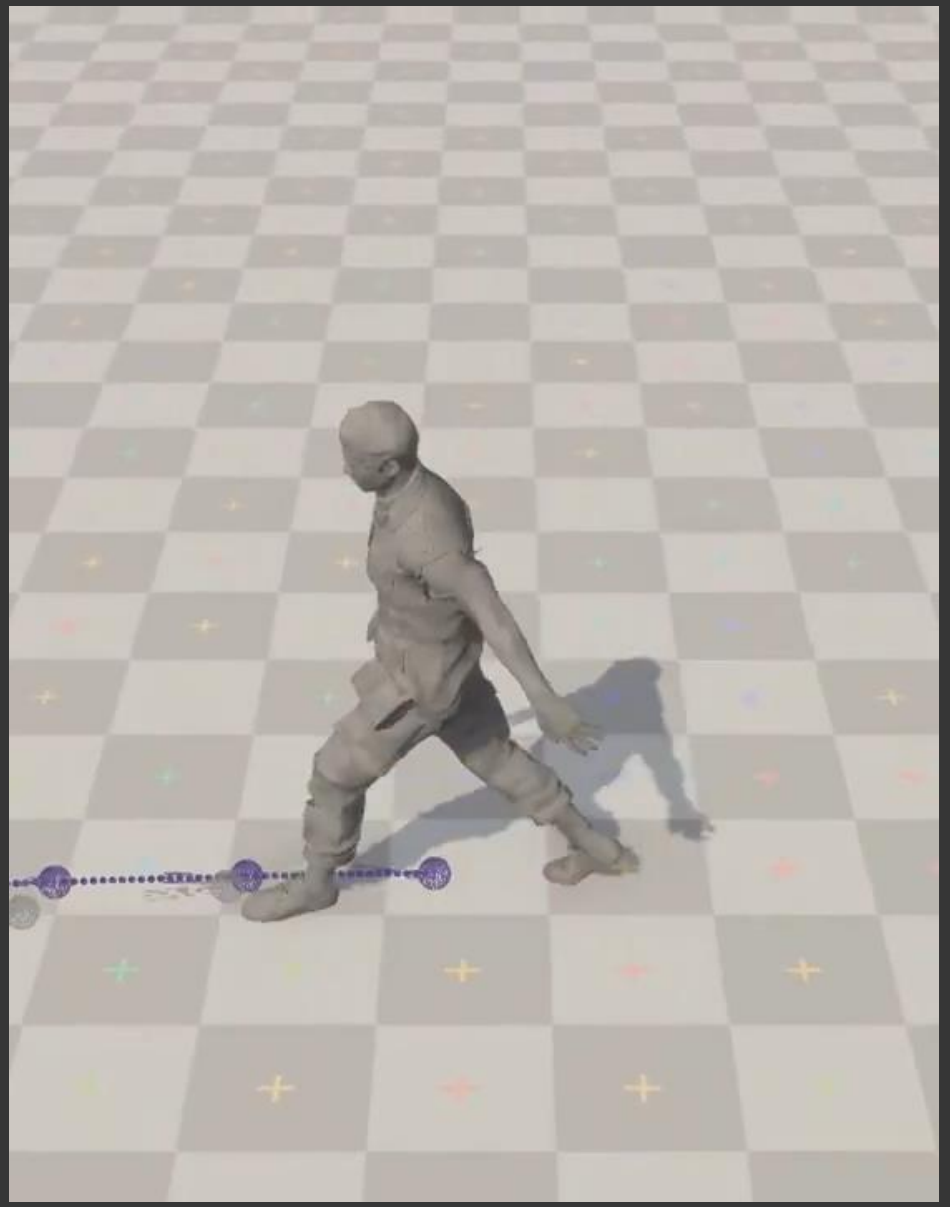
- Brute force search extremely slow
  → use acceleration structure
  - KD-Tree [Clavet 2016]
  - Voxel-based lookup table [Büttner 2018]
  - Clustering [Yi and Jee 2019]
  - Bounding Volume Hierarchy of AABB [Holden et al, 2020]

  - PCA can also be applied to the Feature Database X and feature vectors $\hat{x}$ to reduce their dimensionality
    - Useful way to reduce memory usage and increase performance
    - At the cost of some search accuracy when a large number of redundant or correlated features are used

Results

← Blending Disabled

Blending Enabled →



From [Holden et al. 2020] https://www.youtube.com/watch?v=16CHDQK4W5k

Motion Matching: Conclusion

- A brute force approach
  - Inspired from Motion Graphs
  - Made possible by the computational power available today
  - But memory usage scales linearly with amount of data used

    → trade-off between visual quality, computation time, and production budgets

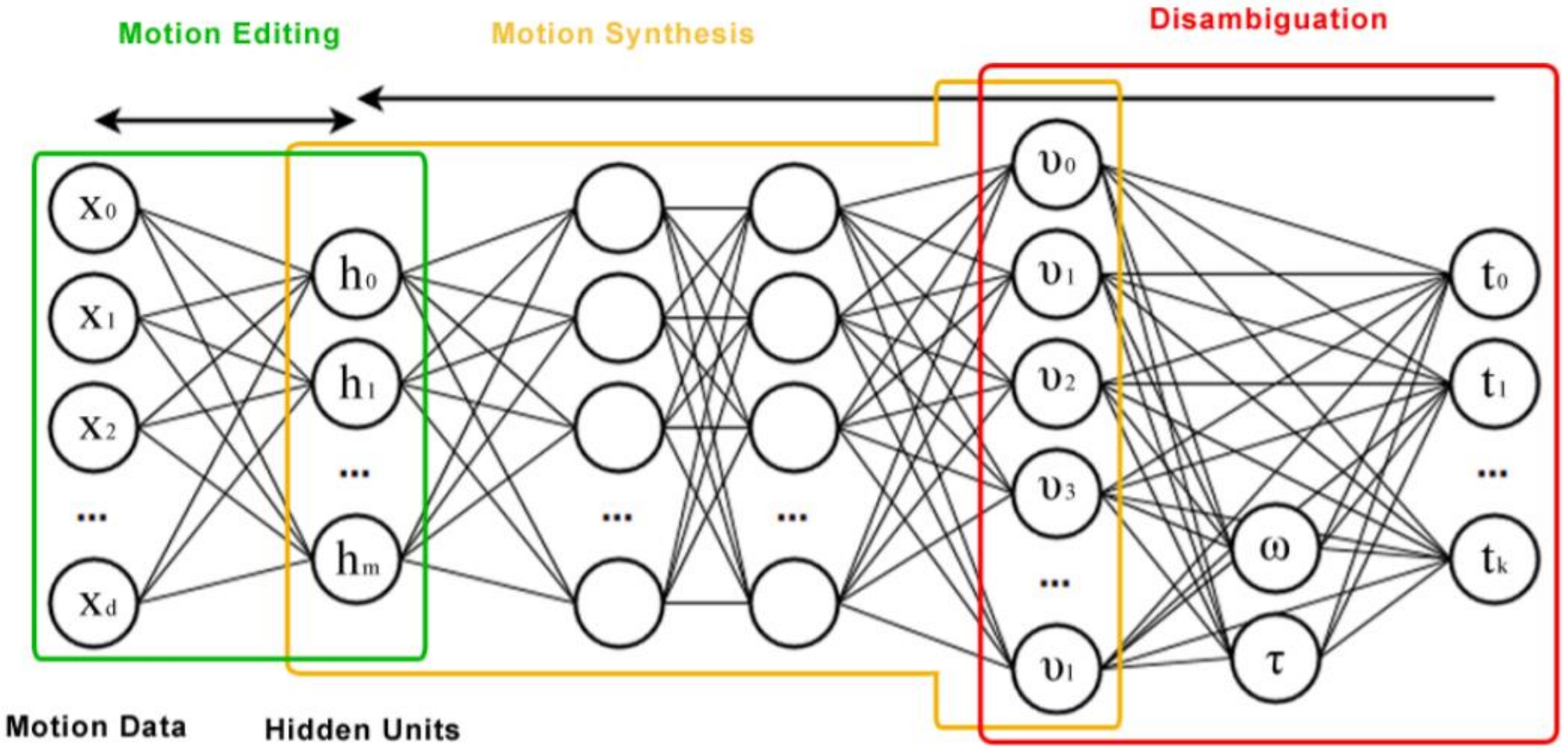
- Almost became a standard approach in the industry since its first use in For Honor (Ubisoft, 2016)

Deep Learning-based Approches

DL approaches

- Recent interest in the Character Animation community for DL-based approaches
  - Since ~2015 with Daniel Holden's work
  - Overall idea: DL can help to learn manifolds of human motion
    - ~ How human motion behaves in a much lower dimension space

DL approaches

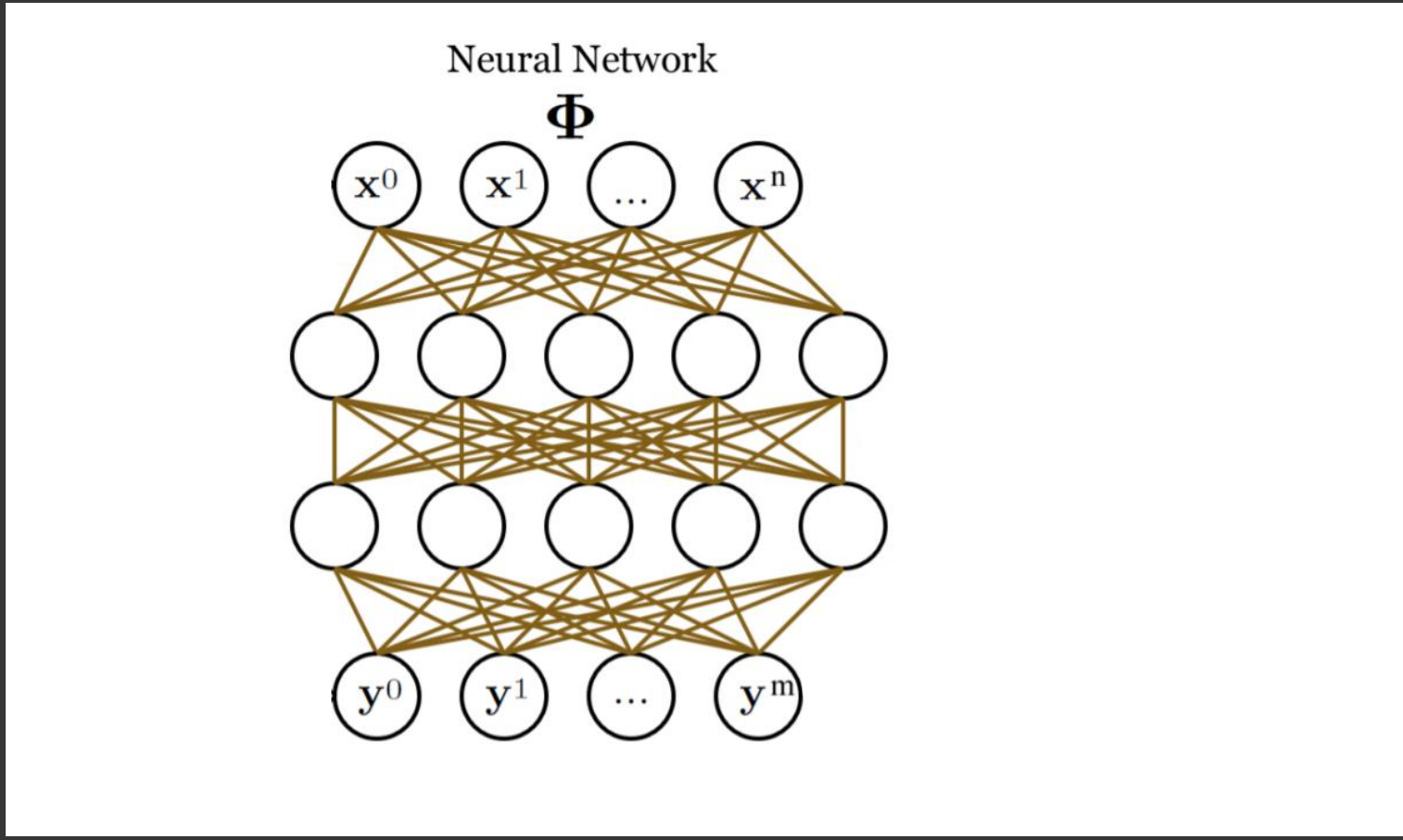


Holden et al. 2016: A Deep Learning Framework for Character Motion Synthesis and Editing

Control Parameters

Phase-Functioned Neural Network.

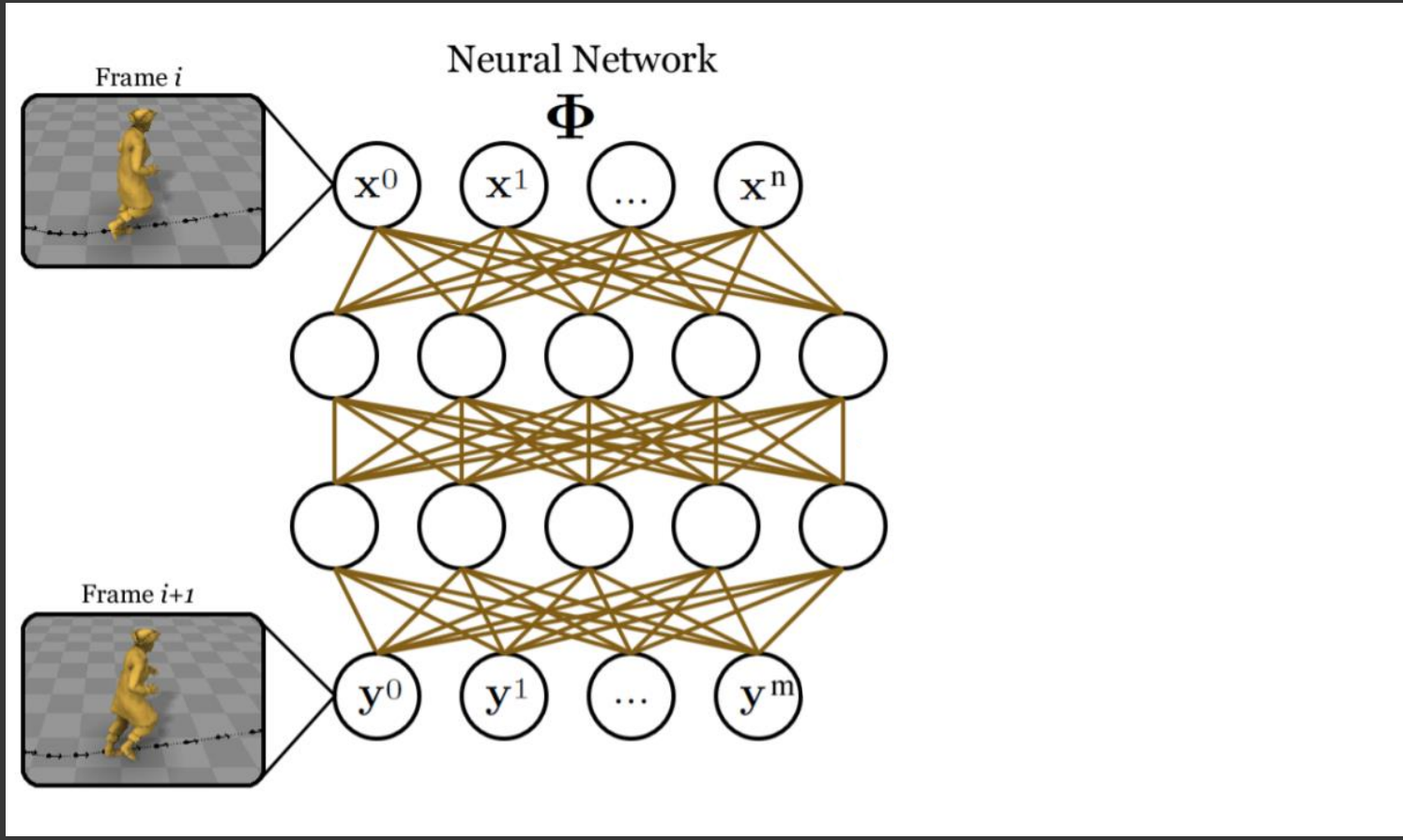
- For interactive control (online control)
  - RNN more appropriate



Holden et al. 2017. Phase-functioned neural networks for character control

Phase-Functioned Neural Network.

- For interactive control (online control)
  - RNN more appropriate
  - But often tend to "die out" when frames of different phases are erroneously blended together

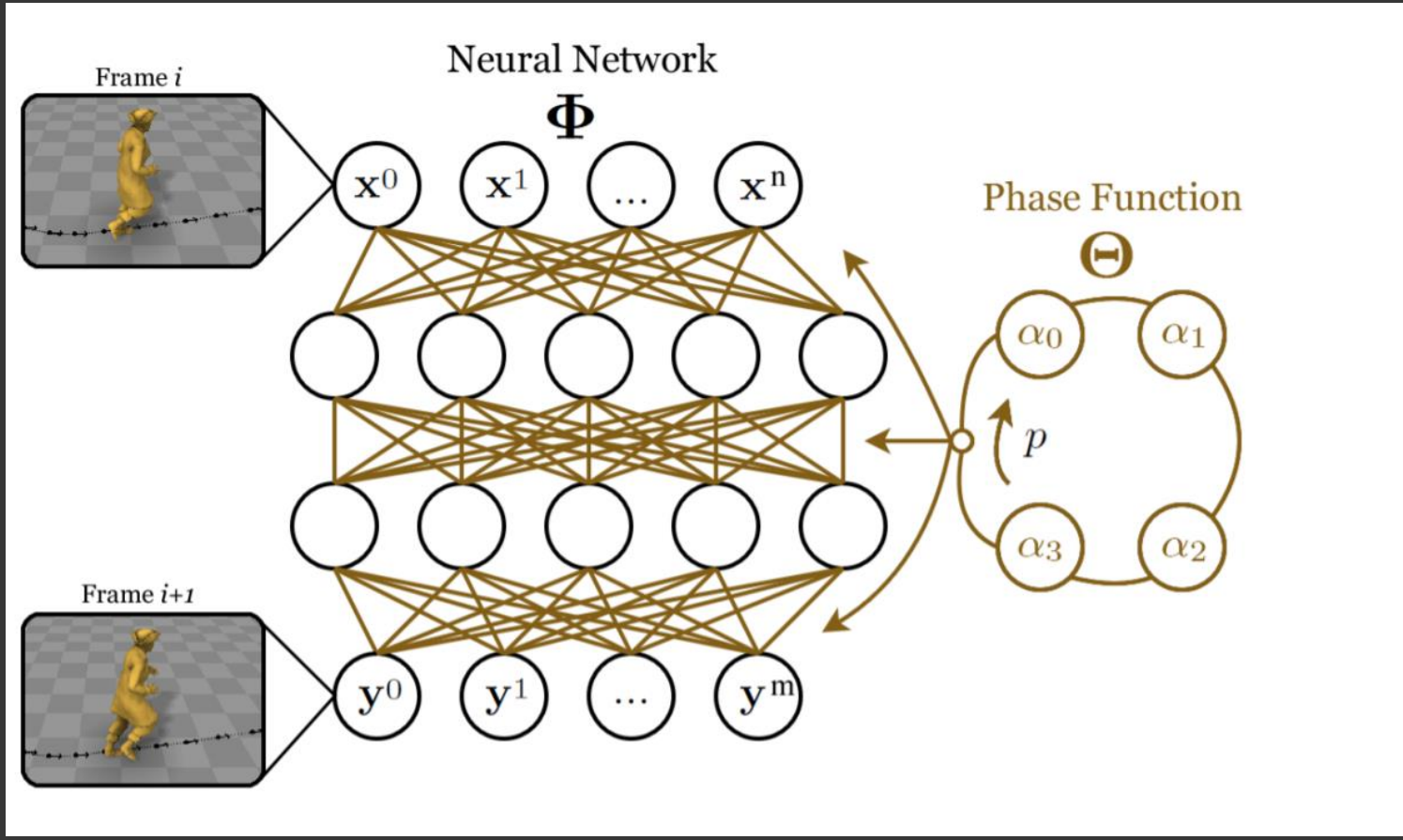


Holden et al. 2017. Phase-functioned neural networks for character control

Phase-Functioned Neural Network.

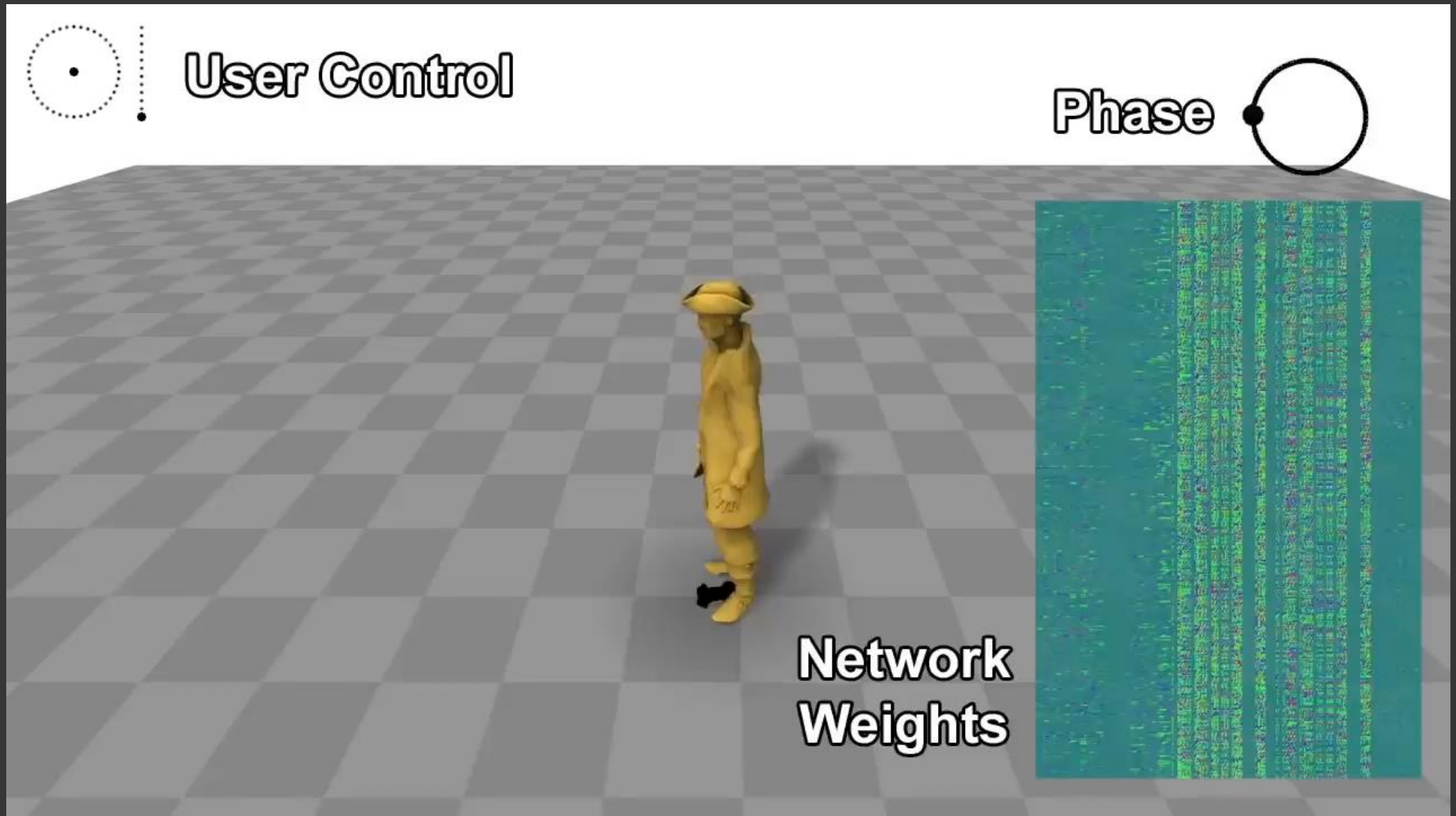
- For interactive control (online control)
  - RNN more appropriate
  - But often tend to "die out" when frames of different phases are erroneously blended together

→ Learn weights of RNN at each frame as a function of the phase



Holden et al. 2017. Phase-functioned neural networks for character control

Phase-Functioned Neural Network.



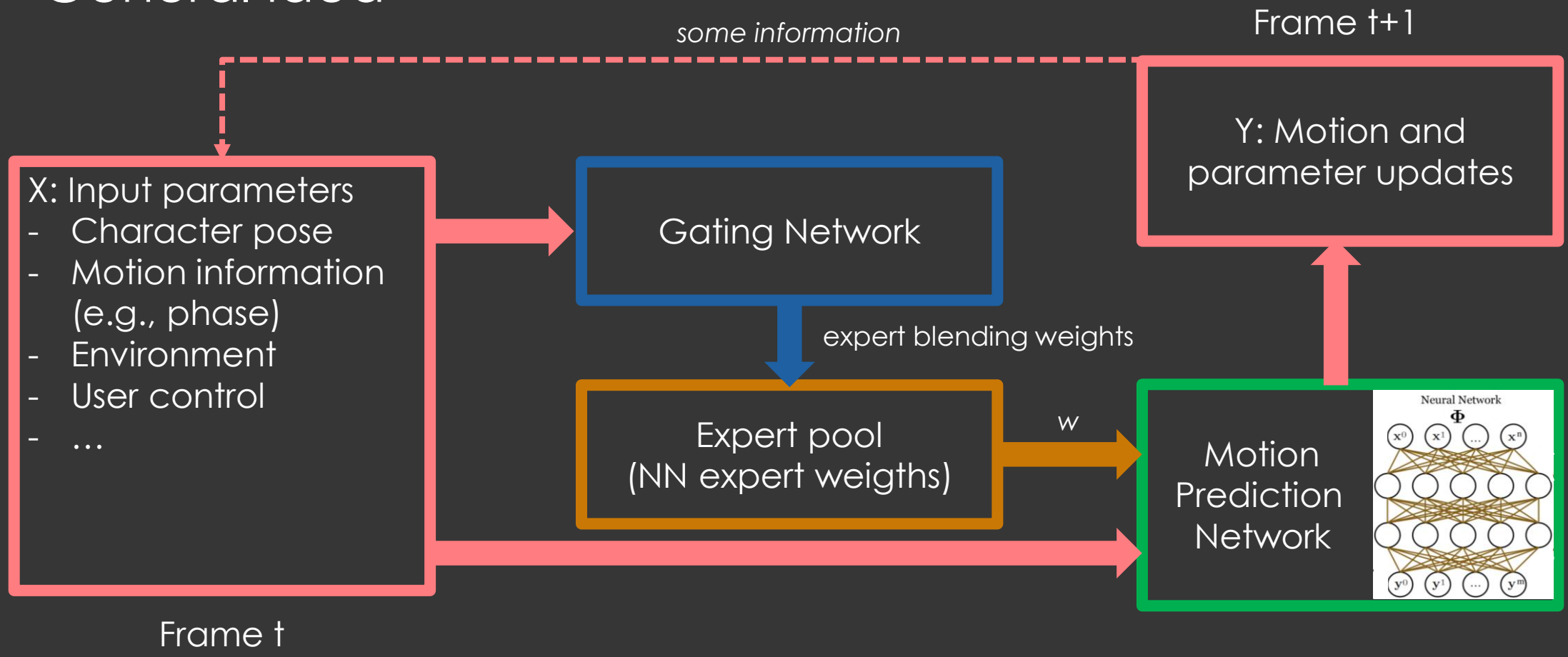
Phase-Functioned Neural Network.

- Limitations of PFNN: phase needs to be manually define
  - Can be complex, or even impossible sometimes

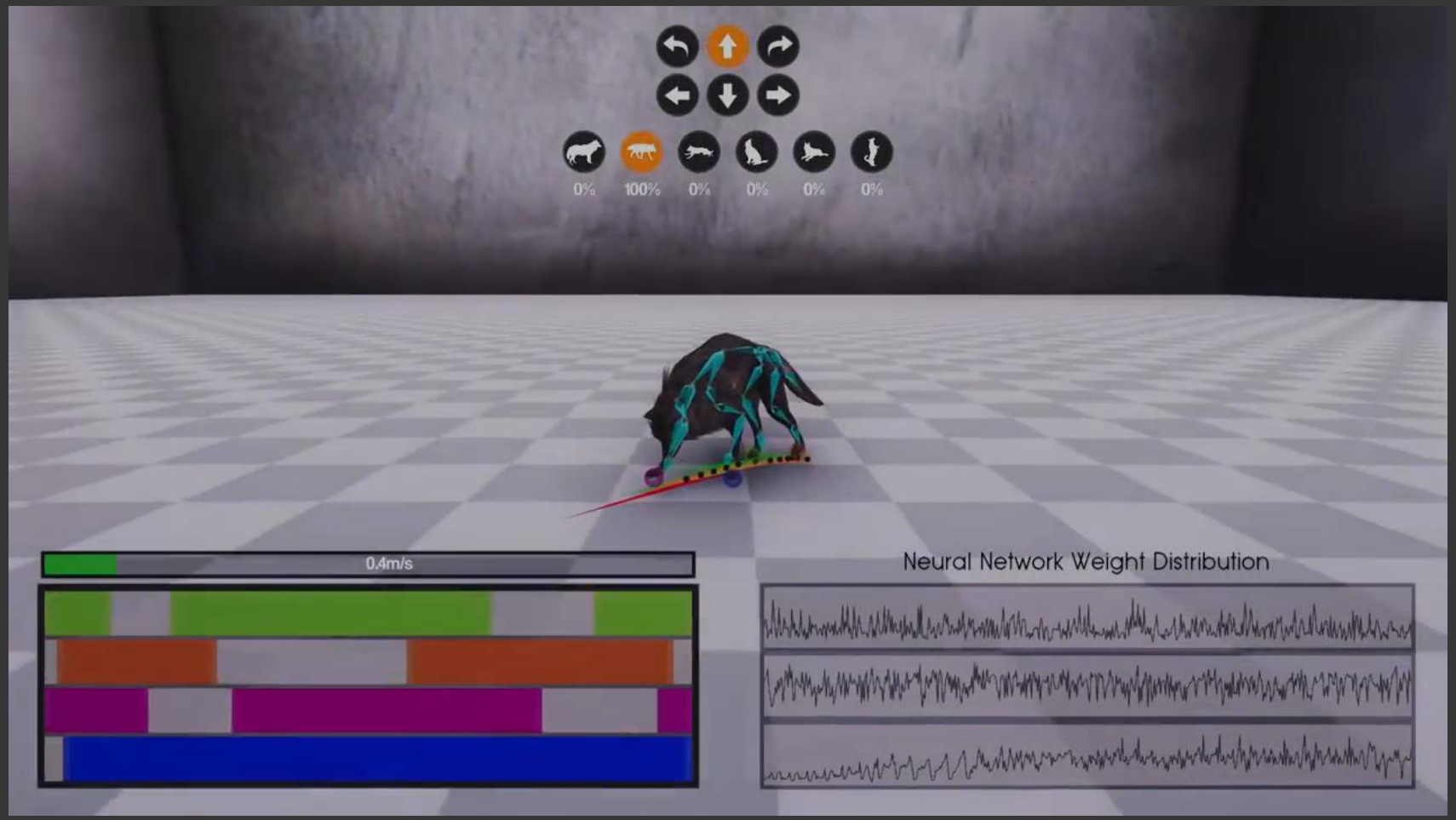
→ Other authors proposed to rely on a mixture-of-experts scheme to dynamically compute the network weights

Mixture-of-experts Schemes

- General idea

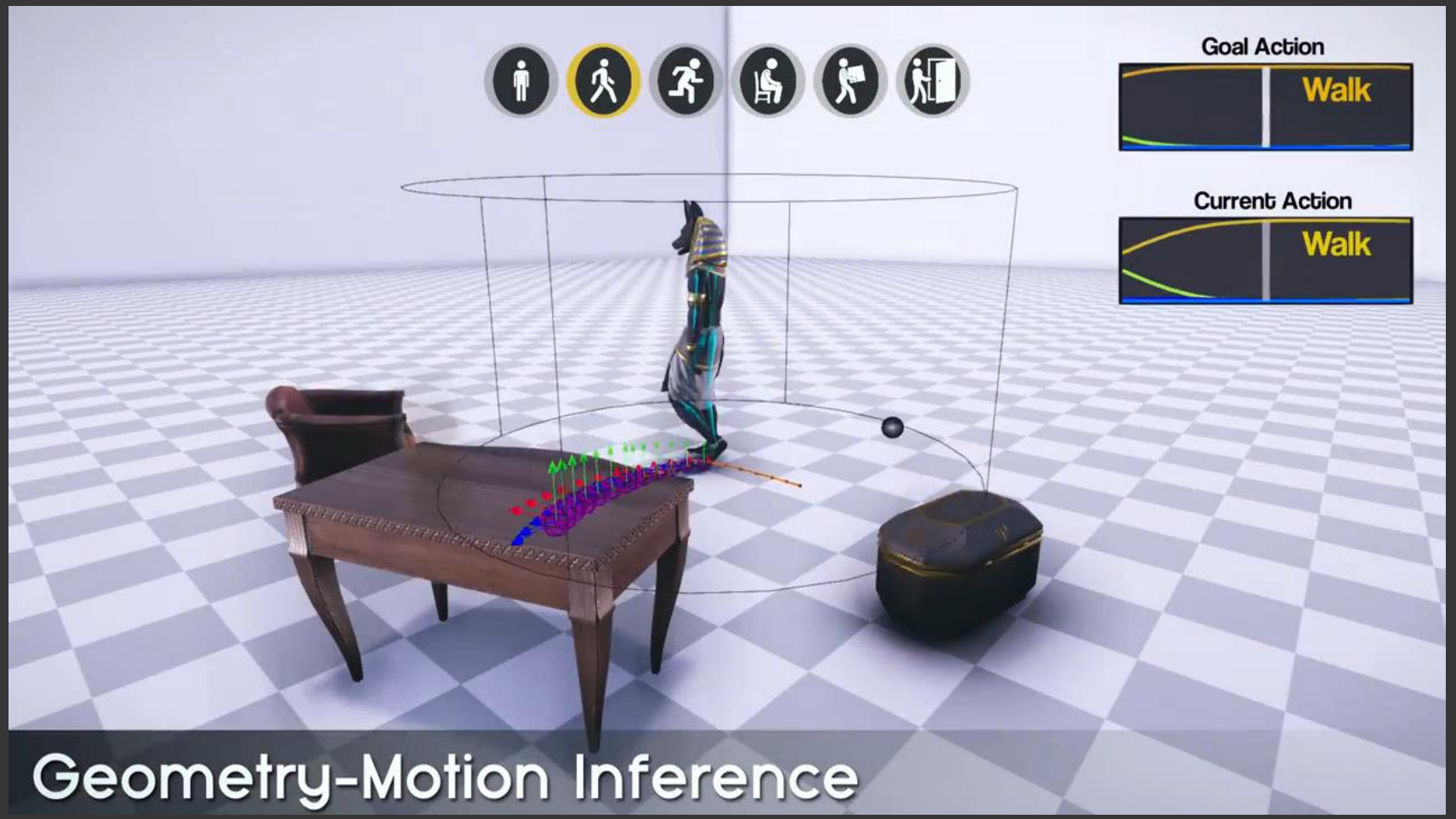


Mixture-of-experts Schemes



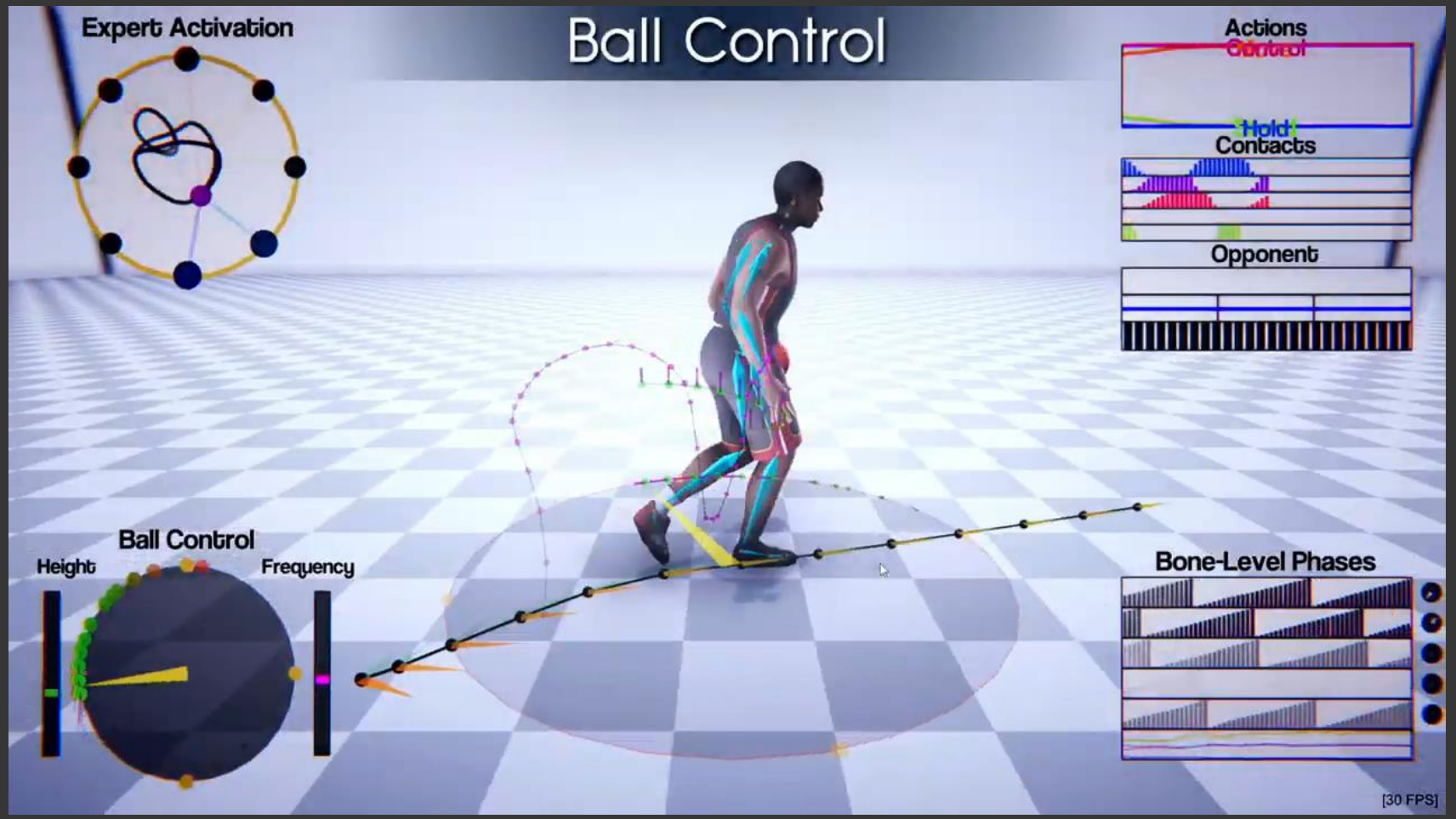
Zhang et al. 2018. Mode-Adaptive Neural Networks for Quadruped Motion Control

Mixture-of-experts Schemes



Starke et al. 2019. Neural State Machine for Character-Scene Interactions

Mixture-of-experts Schemes



Starke et al. 2020. Local Motion Phases for Learning Multi-Contact Character Movements

Other Applications

- More and more work in character animation research using deep learning for
  - Motion prediction and classification
  - Simplifying the motion capture cleaning process
  - Learning alternatives for complex algorithms
    - e.g., Learned Motion Matching
  - Physics-based simulation

Conclusions

Conclusion

- Where does that leave the character?
  - Gaming industry still mostly relies on FSM (Artist control, visual quality)
    - Basic motion graphs give good motion but poor control
    - Other approaches also exit (e.g., optimal control, physical controllers)
      - explored in research, little used to my knowledge in industry
  - General trend in research and industry to go towards Machine Learning-based approaches, as well as physics-based
  - Remember that what remains important for the artist/game designer is to have control over the result